

Ставрополь, 2026 г.

**Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»**

Институт перспективной инженерии

Департамент цифровых, робототехнических систем и электроники

Направление подготовки 09.03.04 Программная инженерия

Направленность (профиль) «Разработка и сопровождение программного обеспечения»

«УТВЕРЖДАЮ»

И.о. директора департамента цифровых,
робототехнических систем и электроники

И. В. Азаров

подпись, инициалы, фамилия

" ____ " ____ 20__ г

**ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ
(ДИПЛОМНЫЙ ПРОЕКТ)**

Студент Душин Александр Владимирович группа ПИЖ-б-о-22-1

1. Тема Разработка веб-приложения для электронных торгов

Утверждена распоряжением по институту № 07.2-Р-12.00 от "26" января 2026 г.

2. Срок представления работы к защите «08» июня 2026 г.

3. Исходные данные для проектирования Требования к ВКР и порядку их выполнения

4. Содержание пояснительной записки:

4.1 Аналитический раздел

4.2 Проектный раздел

4.3 Экономический раздел

4.4 Безопасность и экологичность проекта

4.5 Другие разделы проекта

5 Перечень графического материала (с точным указанием обязательных чертежей)

Дата выдачи задания

Руководитель проекта

подпись

Ф.В. Самойлов

инициалы, фамилия

Консультанты по разделам:

Аналитический раздел

подпись

Е.Н. Новикова

инициалы, фамилия

Проектный раздел

подпись

Е.Н. Новикова

инициалы, фамилия

Экономический раздел

подпись

А.Ю. Орлова

инициалы, фамилия

Безопасность и экологичность

подпись

Е.Н. Новикова

инициалы, фамилия

Нормоконтроль

подпись

А.Ю. Орлова

инициалы, фамилия

Задание принял к исполнению

" ____ " ____ 20__ г.

подпись

**Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»**

Институт перспективной инженерии

Департамент цифровых, робототехнических систем и электроники

Направление подготовки 09.03.04 Программная инженерия

Направленность (профиль) «Разработка и сопровождение программного обеспечения»

КАЛЕНДАРНЫЙ ПЛАН

Фамилия, имя, отчество (полностью) Душин Александр Владимирович

Тема ВКР «Разработка веб-приложения для электронных торгов».

Руководитель: Самойлов Филипп Владимирович

Консультанты: Е.Н. Новикова, А.Ю. Орлова

№	Наименование этапов выпускной квалификационной работы	Срок выполнения работы	Примечание
	1. Выдача задания	26.01.2026	
	2. Начало проектирования	27.01.2026	
	3. Разделы ВКР	27.01.2026-06.06.2026	
	3.1. Аналитический раздел	27.01.2026-15.02.2026	
	3.2. Проектный раздел	16.02.2026-24.05.2026	
	3.3. Экономический раздел	25.05.2026-30.05.2026	
	3.4. Безопасность и экологичность проекта	01.06.2026-06.06.2026	
	4. Предзащита	08.06.2026-10.06.2026	
	5. Рецензирование, нормоконтроль, проверка в системе антиплагиат	11.06.2026-13.06.2026	
	6. Ознакомление с отзывом	15.06.2026-16.06.2026	
	7. Сдача ВКР, отзыва в ГЭК	17.06.2026-20.06.2026	
	8. Защита в ГЭК	25-26.06.2026	

Научный руководитель _____ Самойлов Филипп Владимирович
подпись, Ф.И.О.

И.о. директора департамента цифровых,
робототехнических систем и электроники _____ Азаров Иван Валерьевич
подпись, Ф.И.О.

" ____ " _____ 20 ____ г.

СОДЕРЖАНИЕ

СПИСОК ИСПОЛЬЗУЕМЫХ СОКРАЩЕНИЙ.....	10
ВВЕДЕНИЕ.....	13
ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ФОРМИРОВАНИЕ ТРЕБОВАНИЙ К ПРОГРАММНОМУ ОБЕСПЕЧЕНИЮ	16
1.1. Характеристика предметной области	16
1.1.1. Общая характеристика и основные понятия предметной области..	16
1.1.2. Анализ типовых сценариев деятельности целевой аудитории	16
1.1.3. Описание регламентов и нормативной базы.....	17
1.1.4. Выводы по результатам анализа предметной области.....	17
1.2. Исследование существующих программных решений (анализ аналогов)	17
1.2.1. Обзор аналога № 1 – eBay	17
1.2.2. Обзор аналога № 2 – Avito (раздел «Аукционы»)	18
1.2.3. Обзор аналога № 3 – «Мешок».....	18
1.2.4. Обзор аналога № 4 – Au.ru	18
1.2.5. Сравнительная характеристика и определение недостатков существующих решений.....	18
1.2.6. Обоснование актуальности и конкурентных преимуществ собственной разработки	19
1.3. Обзор и обоснование выбора технологического стека	19
1.4. Постановка задачи и концепция решения	21
1.4.1. Формулировка проблемной ситуации	21
1.4.2. Определение цели и задач создания программного продукта	22
1.4.3. Описание концепции будущего решения (ТО-BE)	23
1.4.4. Функциональные границы продукта	23
1.4.5. Допущения и ограничения	23
1.5. Моделирование и спецификация требований к программному обеспечению	23
1.5.1. Функциональное моделирование процесса в нотации IDEF0.....	23

1.5.2. Моделирование потоков данных (DFD)	25
1.5.3. Определение акторов и построение диаграммы вариантов использования.....	26
1.5.4. Детальное описание функциональных требований (спецификация прецедентов)	27
1.5.5. Формулировка нефункциональных требований.....	27
1.6. Моделирование предметной области (концептуальное проектирование данных).....	27
Выводы по главе 1.....	29
ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ ЭЛЕКТРОННЫХ ТОРГОВ.....	30
2.1. Архитектурный стиль и модель C4.....	30
2.1.1. Выбор архитектурного стиля.....	30
2.1.2. Контекст системы (уровень C1)	30
2.1.3. Контейнеры (уровень C2).....	31
2.1.4. Компоненты (уровень C3).....	31
2.2. Реляционная схема базы данных.....	31
2.3. Проектирование REST API	33
2.4. Подсистема реального времени.....	33
2.5. Подсистема завершения торгов.....	34
2.5.1. Защита от снайперских ставок.....	35
2.6. Подсистема рассылки уведомлений.....	36
2.7. Проектирование клиентской части	37
Выводы по главе 2.....	37
ГЛАВА 3. КОНСТРУИРОВАНИЕ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ	39
3.1. Средства разработки и обоснование версий стека	39
3.2. Структура проекта	39
3.3. Реализация модели данных	39
3.3.1. Эволюция схемы через миграции Alembic.....	39
3.3.2. Хранение денег и времени	40

3.3.3. Реализация ключевых моделей.....	40
3.4. Реализация бизнес-логики.....	40
3.4.1. Размещение ставки и защита от конкурентных операций.....	40
3.4.2. Завершение торгов по событию	41
3.4.3. Выбор ведущего планировщика через рекомендательную блокировку	41
3.4.4. Очередь исходящих писем с экспоненциальной задержкой повторов	42
3.4.5. Транзакционная целостность баланса	42
3.4.6. Идемпотентность повторных уведомлений	43
3.5. Реализация REST-интерфейса	44
3.6. Реализация подсистемы реального времени	44
3.7. Реализация подсистемы уведомлений	44
3.8. Реализация клиентской части	45
3.9. Развёртывание программного обеспечения	45
3.10. Применение принципов качественного кода	46
Выводы по главе 3.....	47
ГЛАВА 4. ТЕСТИРОВАНИЕ И ОБЕСПЕЧЕНИЕ КАЧЕСТВА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ	48
4.1. Стратегия тестирования	48
4.2. Модульное и интеграционное тестирование	48
4.2.1. Тестирование конкурентных операций	48
4.2.2. Покрытие кода тестами	48
4.3. Нагрузочное тестирование	49
4.3.1. Конфигурация испытательного стенда.....	49
4.3.2. Анализ выявленного узкого места	50
4.3.3. Эксперимент: стоимость валидации ответа средствами Pydantic....	51
4.3.4. Проекция результатов на промышленную конфигурацию	51
4.3.5. Микрозамер веерной рассылки уведомлений	52
4.4. Анализ угроз и тестирование безопасности.....	54

4.5. Найденные дефекты и анализ качества	55
4.6. Непрерывная интеграция и валидация развёртывания	57
Выводы по главе 4.....	58
ГЛАВА 5. ТЕХНИКО-ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ.....	59
5.1. Описание программного продукта и цели разработки	59
5.2. Позиционирование на рынке	59
5.3. Планирование разработки	59
5.4. Расчёт себестоимости разработки	60
5.4.1. Заработная плата основная	60
5.4.2. Заработная плата дополнительная	60
5.4.3. Отчисления во внебюджетные фонды.....	60
5.4.4. Материальные затраты и амортизация оборудования	60
5.4.5. Накладные расходы	60
5.5. Расчёт цены и прогноз выручки	61
5.6. Оценка экономии за счёт асинхронной архитектуры	61
5.7. Дисконтированные показатели экономической эффективности	62
5.8. Сводные технико-экономические показатели	63
Выводы по главе 5.....	63
ГЛАВА 6. БЕЗОПАСНОСТЬ И ОХРАНА ТРУДА	65
6.1. Анализ опасных и вредных производственных факторов на рабочем месте программиста	65
6.2. Мероприятия по обеспечению безопасности.....	65
6.2.1. Эргономические требования к рабочему месту.....	65
6.2.2. Расчёт искусственного освещения	65
6.2.3. Параметры микроклимата.....	66
6.3. Электро- и пожарная безопасность	66
6.3.1. Пожарная безопасность	66
6.3.2. Электробезопасность	66
6.3.3. Утилизация электронных отходов	66

Выводы по главе 6.....	67
ЗАКЛЮЧЕНИЕ	68
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	71
ПРИЛОЖЕНИЕ Г.....	76
Г.1. app/services/scheduler_election.py – выбор ведущего узла- планировщика.....	76
Г.2. app/services/email_outbox.py – воркер очереди писем	76
Г.3. app/routers/bids.py – атомарная обработка ставки	77
Г.4. app/services/auction_scheduler.py – защита от снайперских ставок	78
Г.5. app/services/websocket_manager.py – широковежание подписчикам .	78
ПРИЛОЖЕНИЕ Д.....	80
Д.1. Регистрация и вход в систему	80
Д.2. Просмотр каталога лотов.....	80
Д.3. Размещение ставки.....	81
Д.4. Создание собственного лота	81
Д.5. Управление аккаунтом.....	82
Д.6. Публичный профиль продавца	82
ПРИЛОЖЕНИЕ Е	83

СПИСОК ИСПОЛЬЗУЕМЫХ СОКРАЩЕНИЙ

БД – база данных.

БЖД – безопасность жизнедеятельности.

ВКР – выпускная квалификационная работа.

ГК РФ – Гражданский кодекс Российской Федерации.

ГОСТ – государственный стандарт.

МУ – методические указания.

НДС – налог на добавленную стоимость.

НК РФ – Налоговый кодекс Российской Федерации.

ОС – операционная система.

ОСН – общая система налогообложения.

ПК – персональный компьютер.

ПО – программное обеспечение.

ПЭВМ – персональная электронно-вычислительная машина.

РФ – Российская Федерация.

СП – свод правил.

СУБД – система управления базами данных.

СанПиН – санитарные правила и нормы.

ТЭО – технико-экономическое обоснование.

ТЭП – технико-экономические показатели.

ФЗ – федеральный закон.

ФОТ – фонд оплаты труда.

ЦБ – Центральный банк.

ACID – Atomicity, Consistency, Isolation, Durability – свойства транзакций СУБД.

API – Application Programming Interface – программный интерфейс приложения.

ASGI – Asynchronous Server Gateway Interface.

ASVS – Application Security Verification Standard – стандарт верификации безопасности приложений.

BID – режим торгов с повышением ставки (английский аукцион).

BIN – Buy It Now – фиксированная цена покупки лота.

C2C – Consumer to Consumer – модель «потребитель – потребителю».

CI – Continuous Integration – непрерывная интеграция.

CORS – Cross-Origin Resource Sharing – политика разделения ресурсов между источниками.

CRUD – Create, Read, Update, Delete.

CVE – Common Vulnerabilities and Exposures.

DCF – Discounted Cash Flow – дисконтированный денежный поток.

DI – Dependency Injection – внедрение зависимостей.

ER – Entity-Relationship – модель «сущность – связь».

GIL – Global Interpreter Lock – глобальная блокировка интерпретатора Python.

GUI – Graphical User Interface – графический интерфейс пользователя.

HTTP – Hypertext Transfer Protocol.

HTTPS – HTTP Secure – защищённая версия HTTP.

IDE – Integrated Development Environment – интегрированная среда разработки.

IETF – Internet Engineering Task Force.

IRR – Internal Rate of Return – внутренняя норма доходности.

JSON – JavaScript Object Notation.

JWT – JSON Web Token.

MPA – Multi-Page Application – многостраничное приложение.

MVC – Model-View-Controller.

MVP – Minimum Viable Product – минимально жизнеспособный продукт.

NPV – Net Present Value – чистая приведённая стоимость.

OPEX – Operating Expenses – операционные расходы.

ORM – Object-Relational Mapping – объектно-реляционное отображение.

OWASP – Open Web Application Security Project.

PI – Profitability Index – индекс доходности.

REST – Representational State Transfer – архитектурный стиль API.

RFC – Request for Comments – серия документов IETF.

SMTP – Simple Mail Transfer Protocol.

SOLID – пять принципов проектирования: SRP, OCP, LSP, ISP, DIP.

SPA – Single-Page Application – одностраничное приложение.

SQL – Structured Query Language.

SSE – Server-Sent Events – однонаправленные серверные события.

SaaS – Software as a Service – программное обеспечение как услуга.

TCO – Total Cost of Ownership – совокупная стоимость владения.

TCP – Transmission Control Protocol.

TLS – Transport Layer Security.

UI – User Interface – пользовательский интерфейс.

UML – Unified Modeling Language – унифицированный язык моделирования.

URL – Uniform Resource Locator – единый указатель ресурса.

UTC – Coordinated Universal Time – всемирное координированное время.

WS – WebSocket – двунаправленный протокол поверх TCP.

XSS – Cross-Site Scripting – межсайтовый скриптинг.

ВВЕДЕНИЕ

Под электронными торгами в настоящей работе понимается аукционная торговля между частными лицами (C2C) по модели английского аукциона, а не электронные процедуры государственных и корпоративных закупок. Российский рынок таких C2C-аукционов перестроился после ухода eBay в 2022 году. Avito, Au.ru и «Мешок» закрыли освободившуюся нишу частично – без полноценного real-time или с ограничением тематики. Спрос на универсальную площадку с обновлением цен по WebSocket сохранился. С инженерной точки зрения аукцион – это веб-приложение с высокой долей конкурентных операций: в последние секунды торгов на одну строку базы одновременно поступают десятки запросов. Каждая ставка должна выполняться атомарно, а её результат должен мгновенно достигать всех участников.

Синхронные веб-стеки на пуле потоков сталкиваются с линейным ограничением: один поток операционной системы обслуживает одно открытое соединение. Асинхронная модель (библиотеки `asyncio`, `FastAPI`, `asyncpg`, `AsyncSession` из `SQLAlchemy` версии 2.x) снимает это ограничение в пределах одного процесса: пока сопрограмма ожидает операцию ввода-вывода, цикл событий переключается на обработку других запросов. Один процесс `uvicorn` удерживает тысячи одновременных WebSocket-соединений без выделения отдельного потока операционной системы на каждое подключение – подобная характеристика недостижима для синхронного стека по требованиям к оперативной памяти. Отсюда теоретическая значимость темы – применимость реализуемых на асинхронном стеке методов обеспечения корректности к широкому классу задач с конкурентным доступом к разделяемому состоянию.

Объект исследования – процесс проведения электронных торгов (онлайн-аукционов) в режиме реального времени с одновременным участием множества пользователей. Предмет исследования – методы и средства автоматизации этого процесса, в том числе технологии асинхронного программирования,

обеспечивающие корректность финансовых операций и доставку обновлений в режиме реального времени.

Цель работы – автоматизация процесса проведения электронных торгов посредством веб-приложения «Лотус» на основе технологий асинхронного программирования, обеспечивающего корректность финансовых операций в условиях высокой конкурентности и доставку обновлений в режиме реального времени. Для достижения цели решаются задачи:

1. анализ предметной области, исследование аналогов, формирование требований;
2. проектирование архитектуры по модели C4, разработка схемы БД и контракта REST API;
3. реализация серверной и клиентской частей с обеспечением транзакционной целостности и защиты от состояний гонки;
4. функциональное, интеграционное и нагрузочное тестирование, оценка пропускной способности;
5. технико-экономическое обоснование и анализ требований безопасности труда.

Применены методы системного и сравнительного анализа, спецификации требований по стандарту IEEE 830-1998 [1], объектно-ориентированного проектирования с нотацией UML (Use Case, ER, Sequence, Component) и моделью C4, модульного и интеграционного тестирования, нагрузочного профилирования с измерением хвостовых перцентилей задержки. Корректность реализации проверена набором из 291 автоматизированного теста на фреймворке pytest [2]. Теоретическую базу выбора асинхронной модели составили работы Hattingh [3] и Ramalho [4].

Практическая значимость – готовое к развёртыванию веб-приложение с измеренным профилем нагрузки на программном интерфейсе каталога (раздел 4.3) и подтверждённой способностью обслуживать тысячи одновременных подписчиков WebSocket на один процесс. Сопутствующий результат – пять

переносимых архитектурных решений, применимых за пределами аукционной тематики (раскрыты в заключении).

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ФОРМИРОВАНИЕ ТРЕБОВАНИЙ К ПРОГРАММНОМУ ОБЕСПЕЧЕНИЮ

1.1. Характеристика предметной области

1.1.1. Общая характеристика и основные понятия предметной области

Электронный аукцион имеет простое устройство: продавец выкладывает товар на торги, покупатели поднимают цену, победителем признаётся участник с наивысшим предложением. Объектом исследования служит универсальная С2С-площадка – здесь любой пользователь одновременно может выступать и продавцом, и покупателем. Ключевые понятия:

- лот (auction) – единица торгов: описание, стартовая и текущая цена, временные границы, победитель на момент завершения;
- ставка (bid) – заявка участника по цене, строго превышающей текущую;
- выкуп по фиксированной цене (buy-it-now, BIN) – альтернативный механизм завершения сделки без торгов;
- снайпинг (sniping) – практика максимальной ставки в последние секунды; типовой сценарий, под который проектируется интерфейс.

С точки зрения программной инженерии интернет-аукцион принадлежит к классу веб-приложений с высокой долей конкурентных операций над общим состоянием. В минуты, близкие к закрытию торгов, на одну строку базы поступают десятки запросов. Каждый должен выполняться атомарно. Отсюда два инженерных приоритета – управление параллельным доступом на стороне СУБД и отзывчивость интерфейса через асинхронную модель веб-сервера.

1.1.2. Анализ типовых сценариев деятельности целевой аудитории

Разработка ведётся в инициативном порядке без привязки к заказчику. Отправная точка для требований – открытые отзывы пользователей действующих площадок. Выделяются три сегмента: коллекционеры; продавцы-

любители; охотники за выгодой. Финальные секунды торгов должны отрабатываться без сбоев – типовая претензия пользователей крупных площадок: интерфейс не успевает обновиться, и пользователь видит устаревшую цену, уже перебитую другими ставками. Требование транслируется в архитектуру – push через WebSocket поверх асинхронного веб-сервера.

1.1.3. Описание регламентов и нормативной базы

Деятельность интернет-аукционов в РФ регулируется общими нормами гражданского права: ГК РФ статьи 447–449 (торги) [5], Закон «О защите прав потребителей» [6], ФЗ № 152-ФЗ «О персональных данных» [7]. Расчёты ведутся в условных внутренних единицах – это выводит продукт из-под ФЗ № 161-ФЗ «О национальной платёжной системе»; лицензирование не требуется.

1.1.4. Выводы по результатам анализа предметной области

Анализ показал ключевую особенность интернет-аукциона: высокая доля конкурентных обращений к разделяемому состоянию в моменты, близкие к закрытию торгов. Бизнес-требования транслируются в технические – асинхронная модель обработки запросов, атомарные транзакции, двусторонний канал для real-time событий.

1.2. Исследование существующих программных решений (анализ аналогов)

1.2.1. Обзор аналога № 1 – eBay

eBay – крупнейший в мире интернет-аукцион, Сан-Хосе, 1995. Поддерживает английский аукцион, режим выкупа BIN (buy-it-now), прокси-биддинг, рейтинг продавцов. Сильные стороны – аудитория и защита сделок. Недостатки – комиссия 10-13%, уход с российского рынка в 2022 году, обновление страницы лота через периодический опрос сервера, а не через push-уведомления.

1.2.2. Обзор аналога № 2 – Avito (раздел «Аукционы»)

Avito расширил платформу разделом аукционов в 2023 году. Достоинства – бесплатное размещение, узнаваемость, мобильное приложение, встроенный мессенджер. Недостатки фундаментальные: аукционный механизм надстроен поверх доски объявлений – без жёстких временных рамок и без автоматического определения победителя.

1.2.3. Обзор аналога № 3 – «Мешок»

«Мешок» – российская площадка для коллекционеров. Сильные стороны – тематическое сообщество, детальная категоризация. Слабые – ни WebSocket, ни SSE; пользователь перезагружает страницу вручную; современного REST API нет.

1.2.4. Обзор аналога № 4 – Au.ru

Au.ru (ранее 24au.ru) – российская площадка частных объявлений с аукционным разделом, работает с 2008 года. Профиль близок к «Мешку»: коллекционная аудитория, восходящие торги и продажа по фиксированной цене, отдельный формат «торги с 1 рубля». Сильные стороны – давнее тематическое сообщество и бесплатное размещение лотов. Слабые – обновление страницы лота вручную (ни WebSocket, ни SSE), отсутствие публичного REST API, устаревший интерфейс; за успешную сделку удерживается комиссия 10% от суммы.

1.2.5. Сравнительная характеристика и определение недостатков существующих решений

Сравнение проведено по семи критериям: функциональность, механизм real-time, защита от сбойных сценариев, способ резервирования средств, стоимость для участника, русскоязычный интерфейс, сильные/слабые стороны по отзывам. Результат – таблица 1.1.

Таблица 1.1 – Сравнительная характеристика интернет-аукционов

Критерий	eBay	Avito Аукционы	«Мешок»	Au.ru	Лотус (разработка)
Английский аукцион	Да	Частично	Да	Да	Да
Buy-it-now (BIN)	Да	Да	Да	Да	Да
Real-time обновление	Polling	Нет	Нет	Нет	WebSocket
Атомарность ставок	Да	Не применимо	Не подтверждено	Не подтверждено	Postgres FOR UPDATE
Публичный REST API	Да	Нет	Нет	Нет	Да (OpenAPI 3.1)
Комиссия продавца	10–13%	0–10%	5%	10%	7%
Русскоязычный UI	Нет	Да	Да	Да	Да

1.2.6. Обоснование актуальности и конкурентных преимуществ собственной разработки

Конкурентные преимущества «Лотуса» – real-time обновления через WebSocket, публичный API на OpenAPI 3.1, обе модели торгов (BID и BIN) и открытая технологическая база. Слабые стороны MVP – отсутствие узнаваемого бренда и мобильного приложения – учтены в комиссионной ставке и прогнозе выручки.

1.3. Обзор и обоснование выбора технологического стека

Стек определяется характером нагрузки. Аукцион – система с конкурентным доступом, атомарностью и немедленной рассылкой событий. Опорой при выборе послужила официальная документация FastAPI [8], SQLAlchemy 2.0 [9], asyncpg [10] и PostgreSQL [11], а также сравнительные исследования концепций конкурентного программирования в Python [3, 4]. Обоснование выбора по четырём подсистемам приведено ниже; итоговая сводка – в таблице 1.2.

Веб-фреймворк. Из трёх вариантов «Django, Flask, FastAPI» выбран FastAPI [8]. Django спроектирован под синхронные ORM-операции и поточную модель wsgi; встроенная асинхронная подсистема существует, но не покрывает

работу с ORM и системой сигналов. Flask лёгок, но не имеет встроенной валидации тел запросов и автогенерации спецификации программного интерфейса. FastAPI построен поверх ASGI-сервера Starlette, штатно поддерживает асинхронные обработчики, генерирует спецификацию OpenAPI 3.1 из аннотаций типов, выполняет валидацию тел запросов средствами библиотеки Pydantic. Эти три свойства ключевые для проекта, работающего в режиме реального времени и предоставляющего публичный программный интерфейс.

Система управления базами данных. Из трёх вариантов «PostgreSQL, MySQL, MongoDB» выбран PostgreSQL 16 [11]. Атомарность финансовых операций требует строгой транзакционной модели ACID и явных блокировок строк через конструкцию `SELECT FOR UPDATE` – обе характеристики реализованы в PostgreSQL без оговорок. Документоориентированный MongoDB не подходит из-за принципиальной потери транзакционной целостности при многоколлекционных операциях. MySQL уступает по поддержке расширенных типов (JSONB, NUMERIC с фиксированной точностью), рекомендательным блокировкам `pg_advisory_lock` и конструкции `SELECT FOR UPDATE SKIP LOCKED`, которые применены в подсистемах координации и очереди писем.

Подсистема реального времени. Из трёх вариантов «длинный опрос, однонаправленные события сервера (Server-Sent Events), протокол WebSocket» выбран WebSocket по RFC 6455 [19]. Длинный опрос даёт задержку в худшем случае равную интервалу опроса и нагружает сервер запросами без полезной нагрузки. Однонаправленные события сервера передают данные только от сервера к клиенту, что не покрывает будущий сценарий двусторонней коммуникации (например, реакции на действия пользователя в аукционном чате). WebSocket даёт двусторонний канал с минимальной задержкой, поддерживается стеком FastAPI и Starlette без дополнительных библиотек.

Планировщик завершения торгов. Из четырёх альтернатив «опрос с фиксированным интервалом, cron, библиотека APScheduler, сопрограмма `asyncio.Task`» выбран четвёртый вариант. Опрос даёт задержку, равную

интервалу опроса, и нагружает СУБД лишними запросами. Cron работает с дискретностью одна минута, что недостаточно для аукциона с секундной точностью закрытия. Библиотека APScheduler добавляет внешнюю зависимость и собственный планировщик, дублирующий цикл событий asyncio. Сопрограмма asyncio.Task через asyncio.sleep интегрирована в цикл событий FastAPI/uvicorn, не требует внешних зависимостей, обеспечивает секундную точность срабатывания и автоматически восстанавливается из БД при перезапуске через хук жизненного цикла приложения lifespan.

Таблица 1.2 – Технологический стек проекта по подсистемам

Подсистема	Технология	Ключевое обоснование
Веб-фреймворк	FastAPI 0.136+	нативный async, Pydantic-валидация, штатная генерация OpenAPI
ASGI-сервер	uvicorn	минимальные накладные расходы при async-исполнении
СУБД	PostgreSQL 16	row-level locks, JSONB, проверенная транзакционная модель
ORM	SQLAlchemy 2.0 (async)	поддержка SELECT FOR UPDATE и AsyncSession
Драйвер БД	asyncpg	один из быстрых async-драйверов для Postgres
Миграции	Alembic	стандарт де-факто для SQLAlchemy
Real-time	WebSocket (Starlette)	двунаправленный, минимальная задержка
Завершение торгов	asyncio.Task	секундная точность без внешних зависимостей
Аутентификация	PyJWT + Argon2id	stateless-токены, OWASP-рекомендованный хеш
Электронная почта	aiosmtplib	не блокирует event loop
Клиентская часть	многостраничный HTML + ванильный JavaScript	без SPA-фреймворка; данные через REST и WebSocket

1.4. Постановка задачи и концепция решения

1.4.1. Формулировка проблемной ситуации

Нишу универсального русскоязычного C2C-аукциона с real-time обновлениями цен после ухода eBay не закрыл ни один из действующих

сервисов. Avito и «Мешок» оставляют пробел по двум параметрам одновременно – WebSocket-push и публичный API.

Без специализированной платформы торги между частными лицами ведутся вручную; их текущее состояние отражает функциональная модель «как есть» (AS-IS) в нотации IDEF0. Контекстная диаграмма (рисунок 1.1) показывает проведение торгов через доски объявлений и мессенджеры: цена обсуждается в личной переписке и по телефону, срок окончания и ход торга держатся на устной договорённости и ручном учёте. Управляющими факторами служат устные правила и доверие сторон, а результатом становятся сделка с задержками, оплата вне платформы и отсутствие истории ставок и гарантий – именно эти недостатки и устраняет автоматизация.

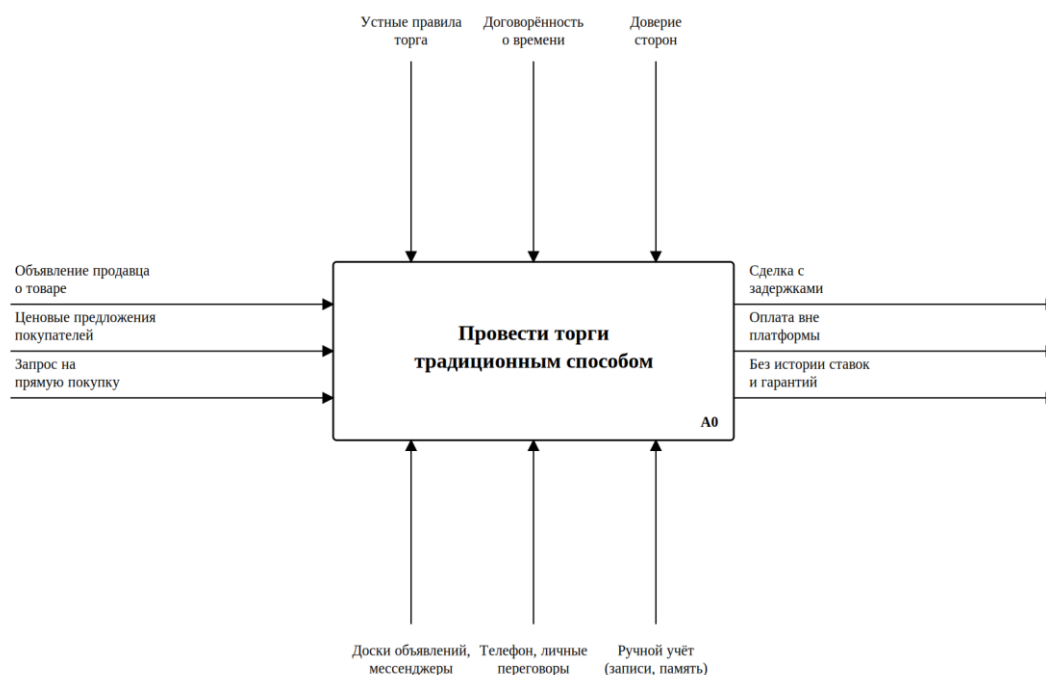


Рисунок 1.1 – Контекстная диаграмма текущего процесса (IDEF0, A-0, AS-IS)

1.4.2. Определение цели и задач создания программного продукта

Цель – автоматизация процесса проведения онлайн-аукционов посредством веб-приложения «Лотус» на асинхронных технологиях, рассчитанного на работу в условиях высокой конкурентности.

1.4.3. Описание концепции будущего решения (TO-BE)

Серверная часть – однопроцессный uvicorn с фреймворком FastAPI, асинхронным соединением с PostgreSQL и WebSocket-каналом. Клиентская часть – многостраничное HTML-приложение без SPA-фреймворка. Координация фоновых задач – через рекомендательную блокировку PostgreSQL. Доставка электронной почты – через очередь outbox.

1.4.4. Функциональные границы продукта

В границах MVP: регистрация и аутентификация; создание и просмотр лотов; ставки в реальном времени; автоматическое завершение торгов; внутренние уведомления; email-уведомления; рейтинговая система и отзывы; подписка на продавцов.

1.4.5. Допущения и ограничения

Расчёты ведутся в условном внутреннем балансе – без интеграции с реальной платёжной системой. Это сознательное ограничение, выводящее MVP из-под ФЗ № 161-ФЗ. Мобильное приложение и полнотекстовый поиск через pg_trgm – направления развития.

1.5. Моделирование и спецификация требований к программному обеспечению

1.5.1. Функциональное моделирование процесса в нотации IDEF0

Целевой процесс, устраняющий недостатки модели AS-IS (рисунок 1.1), описан функциональной моделью «как будет» (TO-BE) в нотации IDEF0. Контекстная диаграмма (рисунок 1.2) сохраняет ту же функцию «Провести электронные торги», но заменяет ручные механизмы программными. Её интерфейсные дуги: входы – заявка продавца на публикацию лота, ставки покупателей и запрос на выкуп по фиксированной цене; управление – правила торгов и валидация, окно анти-снайпинга 120 секунд, ставка комиссии

платформы; механизмы – веб-приложение FastAPI, СУБД PostgreSQL, каналы WebSocket и SMTP; выходы – результат торгов, расчёты по балансу и уведомления участникам.

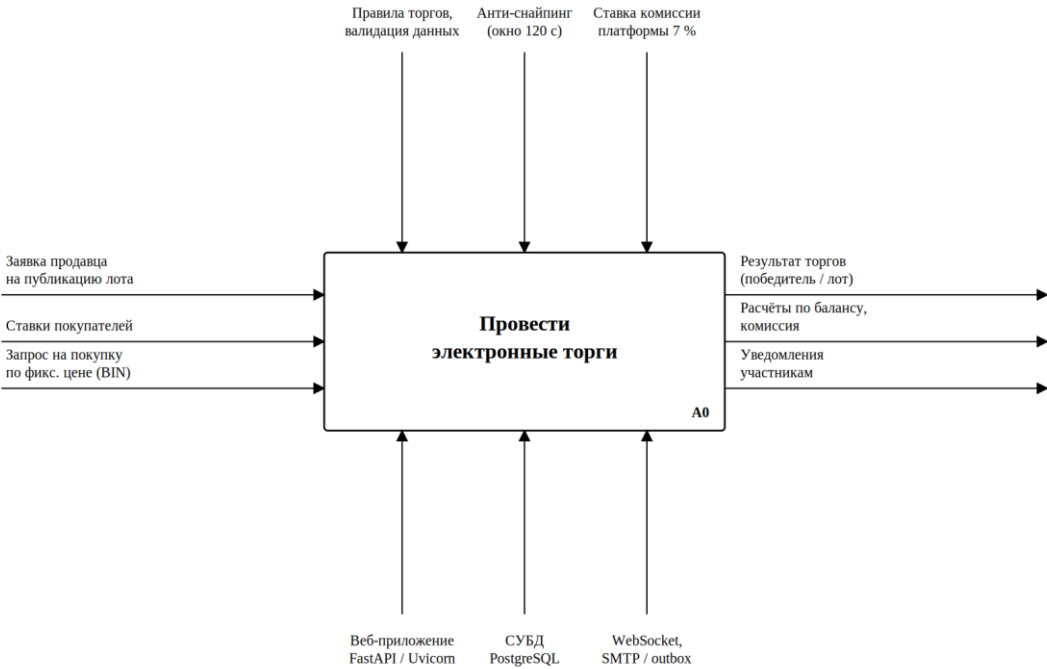


Рисунок 1.2 – Контекстная диаграмма целевого процесса (IDEF0, A-0, TO-BE)

Декомпозиция первого уровня (рисунок 1.3) разбивает целевую функцию на четыре блока: A1 – регистрация и публикация лота, A2 – приём ставок в реальном времени, A3 – завершение торгов и расчёты, A4 – рассылка уведомлений. Выход каждого блока служит входом для следующего: опубликованный лот открывает приём ставок, лидер и текущая цена передаются на завершение, а его итог инициирует уведомления.

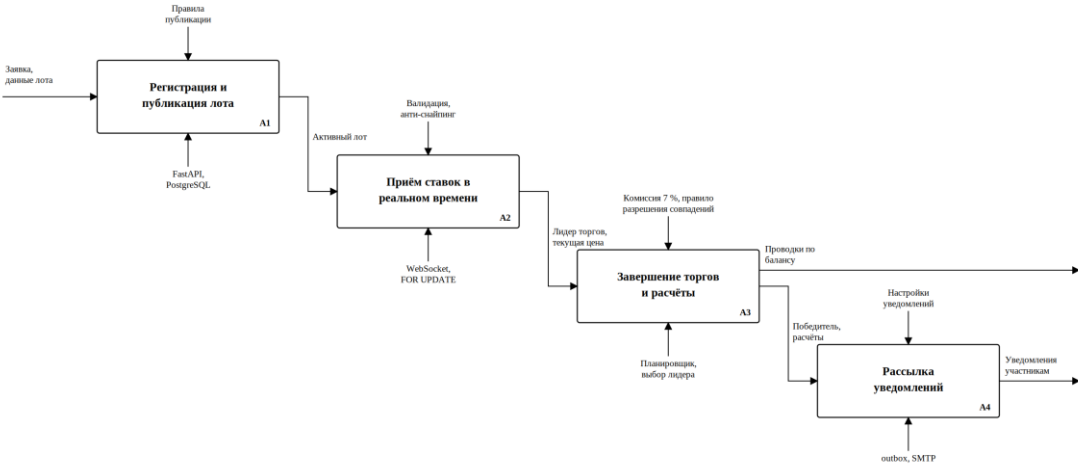


Рисунок 1.3 – Декомпозиция целевого процесса (IDEF0, A0)

Блок A3 несёт наиболее ответственные операции – определение победителя и движение денежных средств, поэтому детализирован до второго уровня (рисунок 1.4). Подблок A31 выбирает победителя по правилу разрешения совпадений; A32 переводит средства продавцу в атомарной транзакции с блокировкой строк участников; A33 удерживает комиссию платформы, записывая проводки в журнал транзакций; A34 формирует итог сделки и инициирует уведомления.

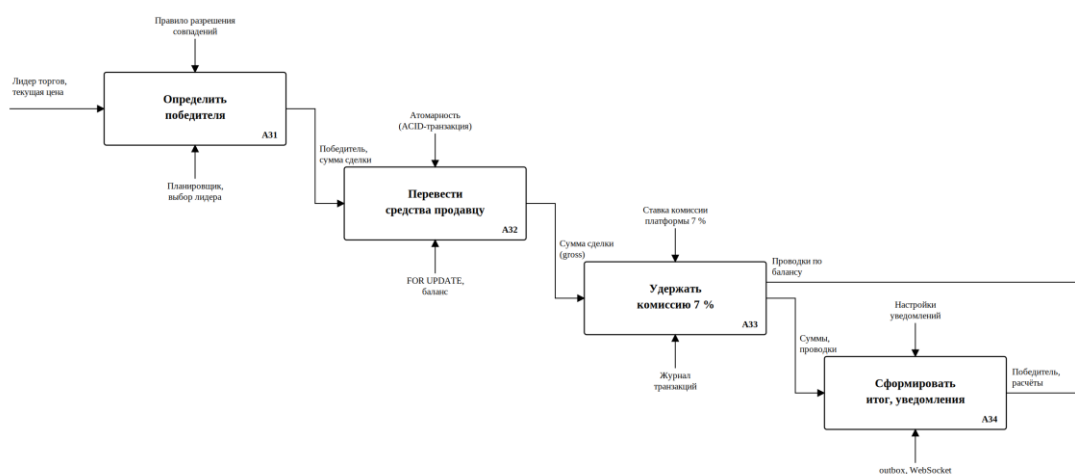


Рисунок 1.4 – Декомпозиция блока «Завершение торгов и расчёты» (IDEF0, A3)

1.5.2. Моделирование потоков данных (DFD)

Движение информации в системе отражает диаграмма потоков данных в нотации Гейна-Сарсона (рисунок 1.5). На ней три внешние сущности – продавец, покупатель и SMTP-сервер; четыре процесса – управление лотами, обработка ставок, завершение и расчёты, рассылка уведомлений; шесть хранилищ D1–D6, соответствующих основным таблицам базы данных: лоты, ставки, пользователи, транзакции, очередь писем и уведомления. Диаграмма прослеживает путь данных от заявки продавца и ставки покупателя до проводок по балансу и доставки писем через очередь outbox.

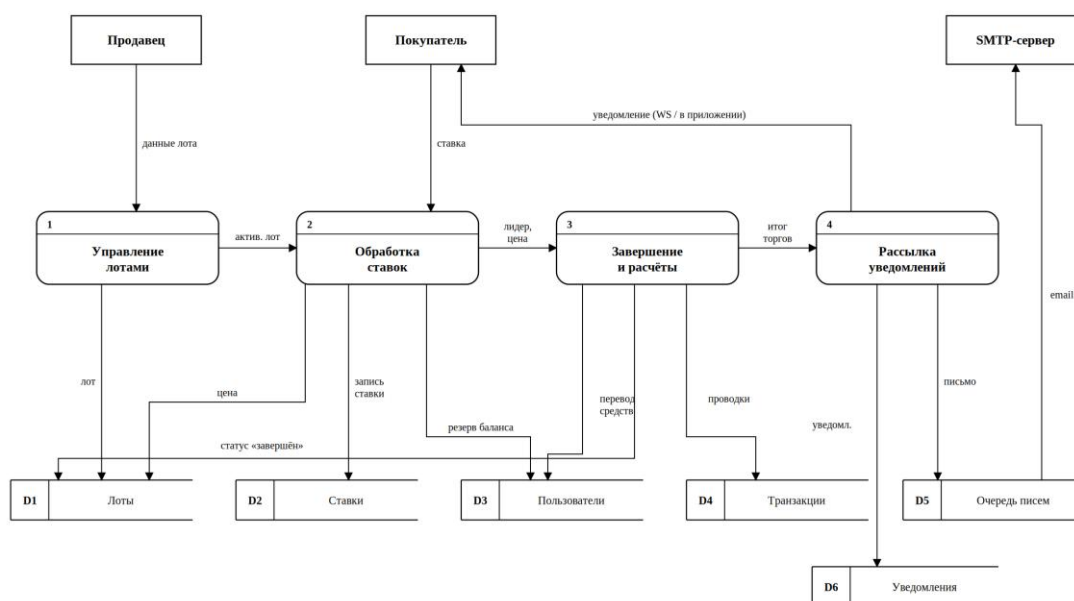


Рисунок 1.5 – Диаграмма потоков данных системы (DFD)

1.5.3. Определение акторов и построение диаграммы вариантов использования

Акторы: «Анонимный посетитель», «Покупатель», «Продавец», «Администратор», «SMTP-сервер», «СУБД PostgreSQL». Диаграмма прецедентов – рисунок 1.6.

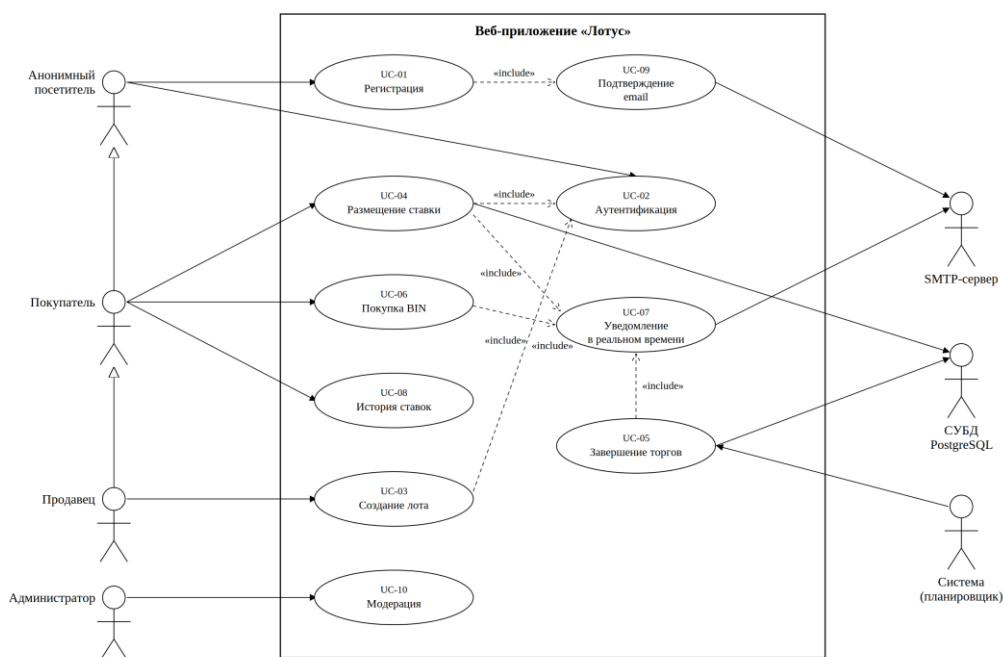


Рисунок 1.6 – Диаграмма вариантов использования системы

1.5.4. Детальное описание функциональных требований (спецификация прецедентов)

Прецеденты описаны по шаблону IEEE Std 830-1998 [1]; сводный перечень приведён в таблице 1.3.

Таблица 1.3 – Сводный перечень функциональных требований

ID	Прецедент	Основной актер
UC-01	Регистрация в системе	Анонимный посетитель
UC-02	Аутентификация по логину и паролю	Зарегистрированный пользователь
UC-03	Создание лота	Продавец
UC-04	Размещение ставки	Покупатель
UC-05	Завершение торгов и определение победителя	Система
UC-06	Покупка по фиксированной цене (BIN)	Покупатель
UC-07	Получение уведомления в режиме реального времени	Покупатель
UC-08	Просмотр истории ставок	Зарегистрированный пользователь

1.5.5. Формулировка нефункциональных требований

Пропускная способность не менее 150 запросов в секунду на программном интерфейсе каталога в конфигурации промышленного развёртывания (gunicorn с четырьмя рабочими процессами) при ста одновременных подключениях. Задержка девяносто девятого перцентиля – не более 3 секунд в той же конфигурации. Удержание не менее 1000 одновременных подписчиков WebSocket на один процесс uvicorn. Восстановление таймеров после перезапуска – автоматическое (через хук жизненного цикла приложения lifespan). Срок жизни access-токена – 2 часа. Все денежные операции – атомарны в пределах одной ACID-транзакции PostgreSQL.

1.6. Моделирование предметной области (концептуальное проектирование данных)

В предметной области электронного аукциона выделены восемь сущностей. «Пользователь» – учётная запись с балансом и флагами

уведомлений; один пользователь одновременно может выступать продавцом и покупателем. «Лот» – единица торгов со стартовой и текущей ценой, временем окончания и категорией. «Ставка» – заявка покупателя на лот с указанием суммы и времени поступления. «Транзакция» – запись о движении средств по балансу пользователя с журналируемым остатком. «Уведомление» – сообщение системы пользователю с пометкой о прочтении. «Очередь писем» – отложенные сообщения для отправки по протоколу SMTP с журналом попыток. «Отзыв» – оценка покупателем продавца после завершённой сделки. «Категория» – иерархический классификатор лотов с возможностью вложенности.

Связи между сущностями выстроены вокруг двух центральных – «Пользователь» и «Лот». Пользователь связан с лотами связью один-ко-многим в двух ролях: как продавец (один пользователь продаёт многие лоты) и как покупатель (один пользователь делает ставки на многие лоты). Лот, в свою очередь, связан с многими ставками и многими транзакциями – резервированием баланса, переводом победителю, удержанием комиссии. Категория самосвязана связью один-ко-многим для иерархии «родитель – потомок». Атрибуты сущностей, их типы и кардинальности детализированы в главе 2 при переходе к физической схеме (раздел 2.2).

Концептуальная модель данных приведена на рисунке 1.7.

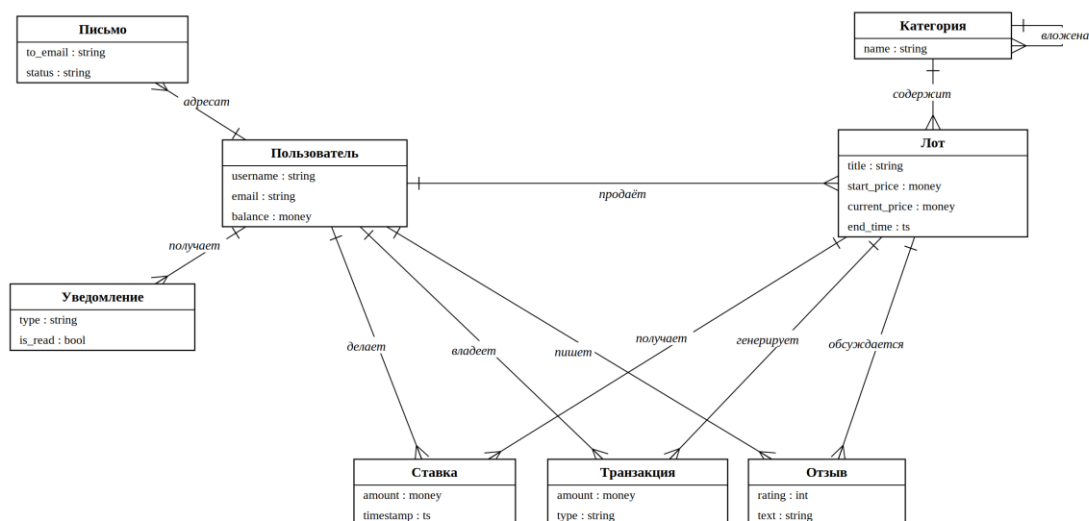


Рисунок 1.7 – Концептуальная модель данных (ER-диаграмма)

Выводы по главе 1

Анализ предметной области определил ключевую инженерную особенность аукциона – конкурентный доступ к разделяемому состоянию в финальные секунды торгов. Сравнение аналогов выявило свободную нишу: универсальная C2C-площадка с WebSocket-push и публичным API. стек определён по четырём подсистемам, требования формализованы по IEEE Std 830-1998. Концептуальная модель данных задаёт основу для проектирования физической схемы в главе 2.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ ЭЛЕКТРОННЫХ ТОРГОВ

2.1. Архитектурный стиль и модель C4

Архитектура «Лотуса» описана по модели C4 Саймона Брауна. Уровень C1 – место сервиса среди внешних пользователей. Уровень C2 – развёртываемые артефакты. Уровень C3 – внутренние модули. Уровень C4 (код) в работе не приводится – в C4-модели он избыточен при наличии листингов ключевых модулей в приложении Г.

2.1.1. Выбор архитектурного стиля

Из трёх вариантов «монолит, микросервисы, событийная архитектура» выбран монолит со слоистым разделением [12]. Приведём три аргумента в пользу выбора:

- объём работ ограничен возможностями одного исполнителя в рамках выпускной квалификационной работы;
- денежные операции укладываются в одну ACID-транзакцию PostgreSQL – шаблон распределённой транзакции Saga неоправдан при нагрузке порядка сотен запросов в секунду;
- асинхронный стек Python в одном процессе удерживает тысячи одновременных WebSocket-сессий без выделения отдельного потока операционной системы на каждое соединение, тогда как синхронному gunicorn для решения той же задачи потребовался бы поток на каждую сессию (см. раздел 4.3) [13].

2.1.2. Контекст системы (уровень C1)

На уровне C1 «Лотус» – чёрный ящик во внешнем окружении. Действующие лица: покупатель и продавец (зарегистрированные пользователи, браузер); анонимный посетитель; администратор. Внешние системы – SMTP-сервер для писем и PostgreSQL для хранения состояния.

2.1.3. Контейнеры (уровень C2)

Четыре артефакта: браузер (HTML/CSS/JS); веб-приложение (uvicorn под FastAPI, HTTPS + WebSocket, фоновые задачи через asyncio); PostgreSQL 16 (TCP); внешний SMTP. Один процесс uvicorn вместо процессного пула – за счёт асинхронной природы FastAPI.

2.1.4. Компоненты (уровень C3)

Запрос проходит через четыре прикладных слоя: представление (templates/, static/); контроллеры (app/routers/); бизнес-логика (app/services/); доступ к данным (модели SQLAlchemy 2.x с асинхронными сессиями). Поверх – четыре долгоживущих компонента координации: планировщик завершения торгов, менеджер WebSocket-соединений, фоновый обработчик очереди исходящих писем, модуль выбора ведущего узла.

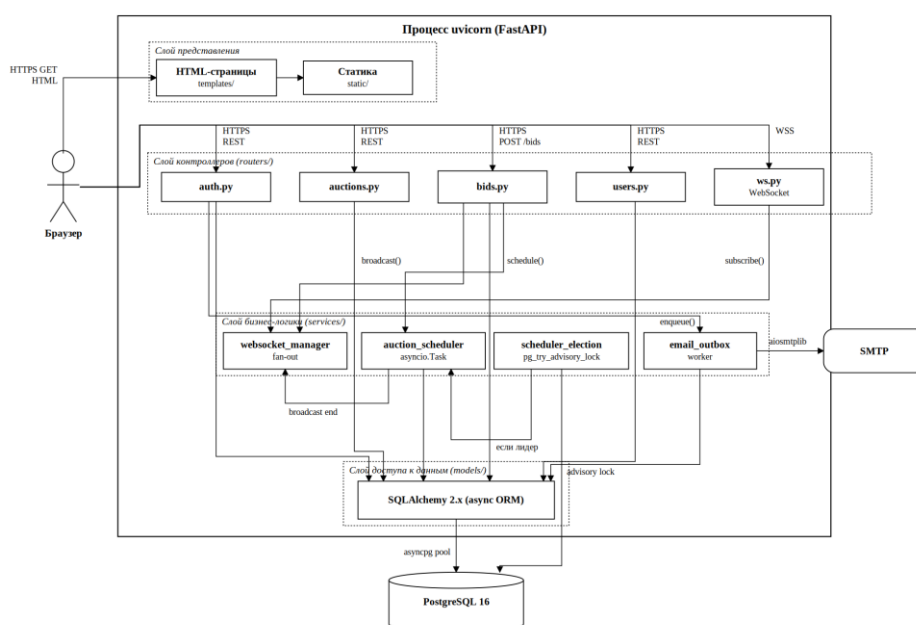


Рисунок 2.1 – Компонентная диаграмма архитектуры сервиса

2.2. Реляционная схема базы данных

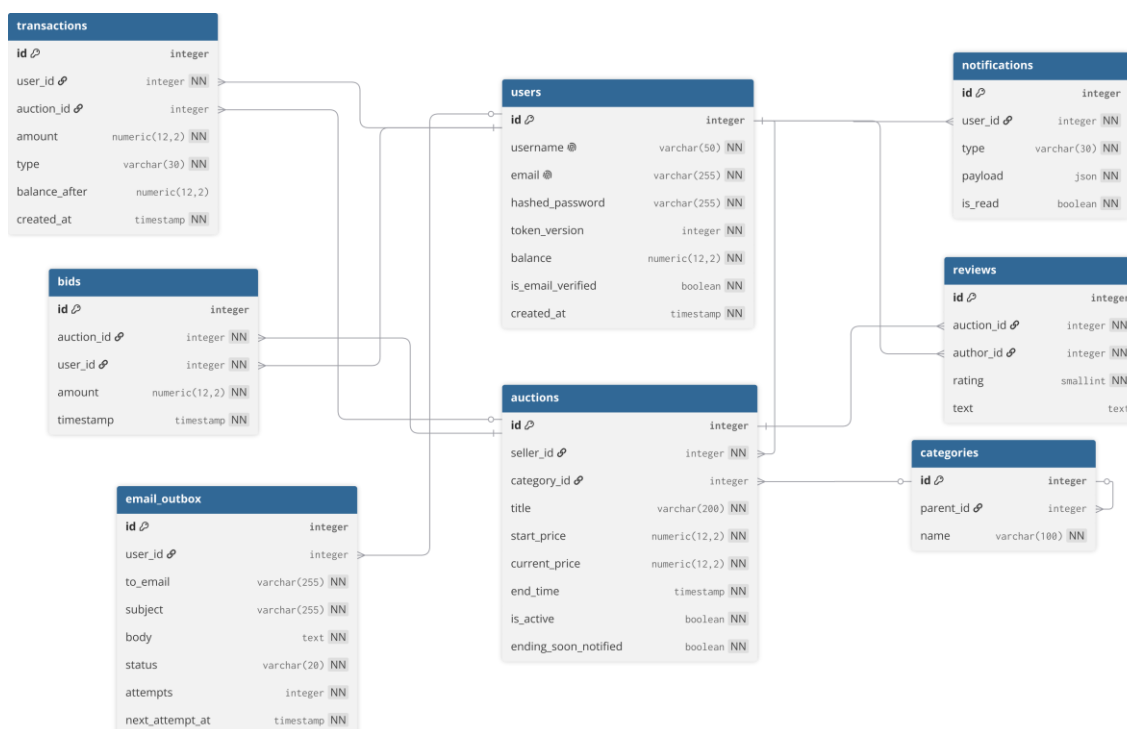
Состояние сервиса хранится в PostgreSQL 16 [11]. Денежные суммы – Numeric(12, 2) [14]: фиксированная точность избегает накопления ошибки

округления double. Временные метки – timestamp с часовым поясом (timestamp_{tz}) в UTC. Идентификаторы – integer с автогенерацией. Подход к схеме данных и индексам опирается на принципы из «Designing Data-Intensive Applications» [15].

Таблица 2.1 – Основные таблицы базы данных

Таблица	Назначение	Ключевые поля
users	Учётные записи	id, username, email, hashed_password, token_version, balance
auctions	Лоты с торгами	id, seller_id, current_price, end_time, is_active
bids	История ставок	id, auction_id, user_id, amount, timestamp
transactions	Журнал движения средств	id, user_id, amount, type, balance_after
notifications	Уведомления пользователей	id, user_id, type, is_read
email_outbox	Очередь исходящих писем	id, to_email, status, attempts, next_attempt_at
reviews	Отзывы по сделкам	id, auction_id, author_id, rating, text
categories	Каталог категорий	id, parent_id, name

Стратегия индексов опирается на типовые запросы: B-tree на внешних ключах; композитный индекс (is_active, end_time) на auctions; индекс (user_id, is_read) на notifications для счётчика непрочитанных. Конкурентный доступ к денежным операциям защищён явными блокировками строк через SELECT FOR UPDATE. Вспомогательная функция lock_users_by_id сортирует идентификаторы по возрастанию – автоматическая защита от взаимной блокировки.



2.3. Проектирование REST API

Контракт описан по OpenAPI 3.1 с использованием семантики HTTP по RFC 9110 [16]. Спецификация автогенерируется из аннотаций обработчиков и Pydantic-схем – расхождение между документом и реализацией исключено конструктивно. Аутентификация – Bearer JWT по RFC 7519 [17] с полями (claims): user_id, tv (token_version), purpose (для специализированных токенов верификации почты и сброса пароля), iss и aud (защита от перепутывания с токенами других сервисов на общем секрете), exp. Валидация тел запросов – через Pydantic [18] с автоматической генерацией OpenAPI-схем. Поле tv – счётчик в БД, инкремент при смене пароля даёт массовую инвалидацию всех сессий одним UPDATE. Срок жизни access-токена – 2 часа.

2.4. Подсистема реального времени

Цена лота меняется в произвольный момент – без потока обновлений в реальном времени пользователь видит устаревшие данные. Из трёх механизмов серверной доставки сообщений (длинный опрос, однонаправленные события

сервера, протокол WebSocket) выбран WebSocket по RFC 6455 [19]. Он уже включён в стек FastAPI и Starlette, даёт единую модель для двух каналов: анонимная лента ставок и аутентифицированная лента уведомлений пользователя.

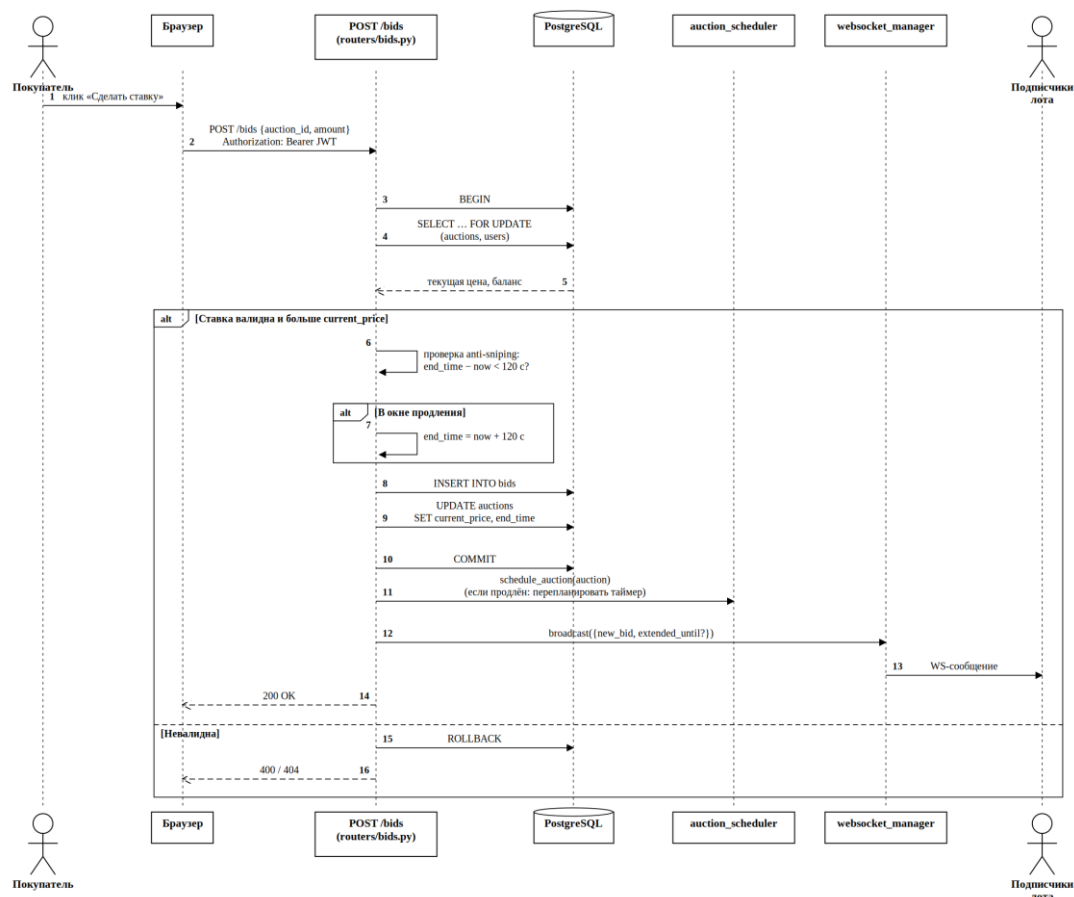


Рисунок 2.3 – Широковещание новой ставки подписчикам аукциона

2.5. Подсистема завершения торгов

Закрытие лота – сопрограмма `asyncio.Task` [20], ожидающая через `asyncio.sleep` до момента `end_time`. Из четырёх альтернатив (опрос, `cron`, `APScheduler`, `asyncio.Task`) выбран четвёртый вариант: секундная точность, отсутствие внешних зависимостей, интеграция в цикл событий FastAPI/uvicorn. Координация в многопроцессном развёртывании – вызов `pg_try_advisory_lock` [21] на старте каждого рабочего процесса. Рекомендательная блокировка принадлежит сессии PostgreSQL: при разрыве соединения она автоматически

освобождается, и при следующем запуске любой из процессов сможет принять роль ведущего.

2.5.1. Защита от снайперских ставок

Снайперская ставка – тактика, при которой участник делает максимальную ставку в последнюю секунду торгов, лишая других участников возможности отреагировать. Тактика широко известна по площадке eBay, где из-за фиксированного времени окончания сложилась культура автоматизированных программ-снайперов. Для устранения недостатка в «Лотусе» реализован механизм автоматического продления: если ставка приходит в последние 120 секунд до конца торгов, новое значение `end_time` устанавливается равным текущему моменту плюс 120 секунд. Окно отсчитывается от момента поступления ставки, а не от старого `end_time` – это даёт всем участникам ровно 120 секунд на реакцию независимо от того, в какой момент закрывающего окна пришла продляющая ставка.

Архитектурно решение опирается на уже существующую инфраструктуру самовосстанавливающегося таймера: после изменения `end_time` планировщик отменяет старую сопрограмму `asyncio.Task` и заводит новую с обновлённым дедлайном. Каждая ставка в окне закрытия торгов продлевает дедлайн, но цепочка ограничена сверху – не более тридцати продлений на лот (около часа); после исчерпания лимита ставки по-прежнему принимаются, а торги закрываются по расписанию. Ограничение защищает от сговора двух участников, которые иначе бесконечно перебивали бы друг друга поздними ставками и удерживали бы лот открытым. Конкурентные ставки в окне продления сериализуются на той же блокировке строки `SELECT FOR UPDATE`, что и обычные ставки. Реализация механизма продления и перепланирования таймера приведена в листинге Г.4 приложения Г.

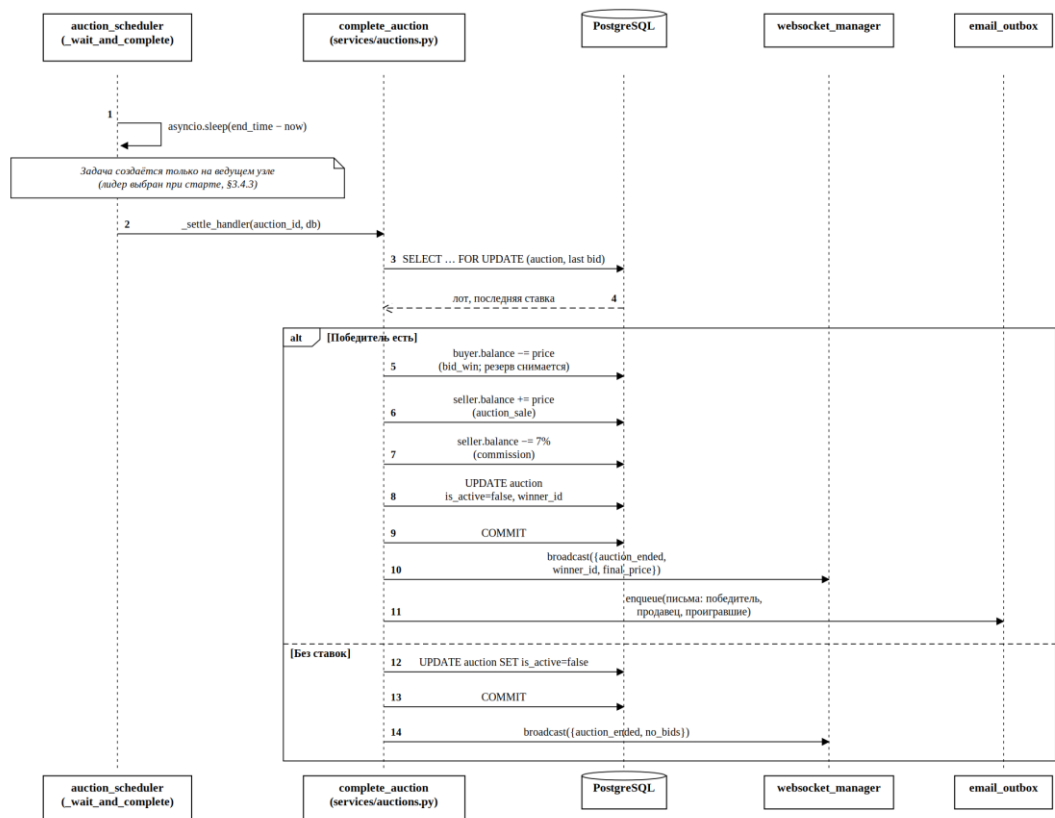


Рисунок 2.4 – Завершение лота с переводом средств

2.6. Подсистема рассылки уведомлений

Три канала: внутрисайтовый (запись в таблицу notifications, доставка гарантирована); WebSocket (моментальная доставка в открытую вкладку браузера); электронная почта (транзакционные письма). Для критически важных писем применён шаблон транзакционной очереди исходящих сообщений (outbox) [22]: HTTP-обработчик вставляет строку в таблицу email_outbox, фоновый обработчик раз в 30 секунд захватывает готовые строки по одной через SELECT FOR UPDATE SKIP LOCKED [23] и отправляет через библиотеку aiomsmtp. При ошибке отправки применяется экспоненциальная задержка повтора 1/5/15/60/360 минут. После исчерпания заданного числа попыток сообщение переводится в очередь отказавших сообщений.

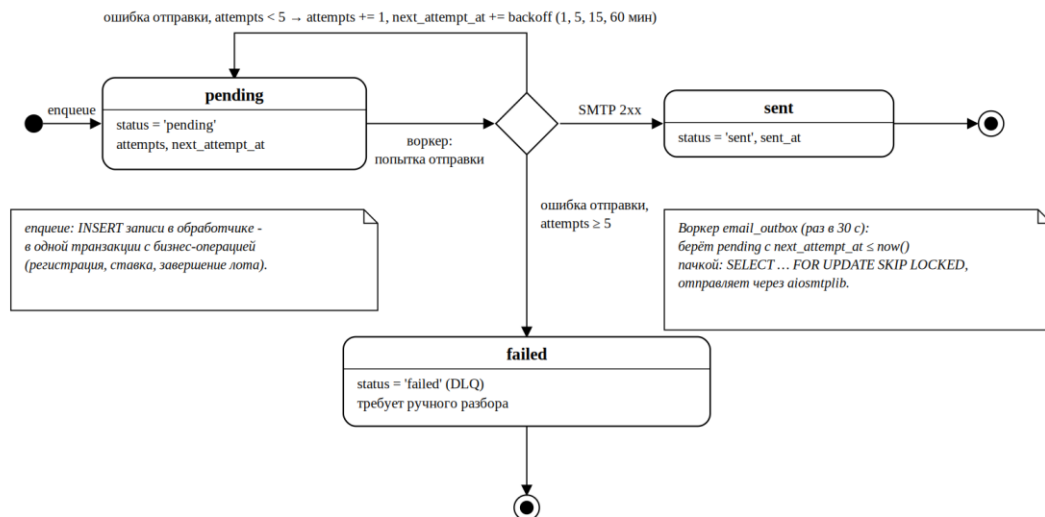


Рисунок 2.5 – Жизненный цикл записи в очереди email_outbox

2.7. Проектирование клиентской части

Пять основных страниц (главная, лот, профиль, список ставок, чужой профиль) и пять служебных (верификация электронной почты, запрос и подтверждение сброса пароля и т.д.). Архитектура – многостраничное приложение без использования SPA-фреймворка. У «Лотуса» нет сценария удержания состояния при переходе между страницами – обновления в реальном времени приходят на открытую страницу через WebSocket. WebSocket-клиент переподключается с экспоненциальной задержкой и применением случайного смещения, что сглаживает пик нагрузки от попыток переподключения при одновременном перезапуске сервера.

Выводы по главе 2

Глава зафиксировала проектные решения, превращающие концептуальную модель из главы 1 в работоспособную архитектуру. Все решения выстроены вокруг асинхронной модели программирования. Архитектура описана по нотации C4 на трёх уровнях. Реляционная схема опирается на тип Numeric для денежных полей и явные блокировки строк для целостности транзакций. Доставка обновлений в реальном времени – через

WebSocket с двумя лентами. Завершение торгов по событию – сопрограмма `asyncio.Task` с координацией через рекомендательную блокировку. Шаблон `outbox` горизонтально масштабируется за счёт конструкции `SKIP LOCKED`. Клиентская часть – многостраничное приложение с обновлениями через WebSocket.

ГЛАВА 3. КОНСТРУИРОВАНИЕ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

3.1. Средства разработки и обоснование версий стека

Версии библиотек закреплены в `requirements.txt` с точностью до уровня патч-версии. Это даёт повторяемость сборки на любой машине разработчика или в CI-окружении. Версии выбраны по трём критериям: отсутствие открытых CVE, наличие `type-hints` в публичном API, поддержка Python 3.11+.

3.2. Структура проекта

Каталог `app/` разделён на пять подпакетов: `routers` (HTTP-маршруты), `services` (бизнес-логика), `models` (модели SQLAlchemy), `schemas` (схемы Pydantic), `utils` (общие утилиты). Каталог `templates/` хранит статические HTML-страницы (данные подставляются на стороне клиента сценариями JavaScript); `static/` – таблицы стилей CSS, сценарии JavaScript, медиафайлы. Миграции базы данных – в `alembic/versions/`. Тесты – в `tests/`; конфигурация `pytest` – в файле `pytest.ini`.

3.3. Реализация модели данных

3.3.1. Эволюция схемы через миграции Alembic

Схема развивалась через восемнадцать ревизий Alembic [24]: начальная схема → `notification types (bid_received, lost)` → внешние ключи на таблицу `auctions` → `token_version` → `email_outbox` → `email_verified` → `password_reset_sent_at` → новый тип транзакции `commission` → счётчик неудачных логинов с блокировкой (`failed_login_count, locked_until`) → `rolling-window` учёта загруженных байт (`upload_bytes_window, upload_window_start`). Завершающая серия ревизий ужесточает инварианты данных на уровне схемы: ЧЕКС-ограничения (`bin_price` не ниже стартовой цены, неотрицательность баланса), счётчик продлений лота, перевод всех временных столбцов на тип

TIMESTAMP WITH TIME ZONE, ограничение длины строковых столбцов, обязательность полей categories.name/slug и reviews.auction_id с индексами под внешние ключи, приведение имён пользователей к нижнему регистру для регистронезависимой уникальности. Каждая миграция протестирована в CI шагом alembic upgrade head до запуска pytest [2].

3.3.2. Хранение денег и времени

Деньги – Numeric(12, 2). Время – timestamp с часовым поясом (timestamptz) в UTC; утилита app.utils.time.utcnow возвращает значение с явной зоной, поэтому сериализованная метка несёт суффикс +00:00 и клиент трактует её как UTC, а не как локальное время браузера. Конвертация в локальное время делается на клиенте по часовому поясу браузера.

3.3.3. Реализация ключевых моделей

Модель User содержит флаги уведомлений (notify_outbid, notify_winning, notify_ending, notify_sold, notify_bid_received, notify_lost), token_version, email_verified, столбцы антибрутфорса (failed_login_count, locked_until) и счётчик rolling-24h byte-квоты на загрузки (upload_bytes_window, upload_window_start). Модель Auction – auction_type ('bid'/'bin') и bin_price.

3.4. Реализация бизнес-логики

3.4.1. Размещение ставки и защита от конкурентных операций

Маршрут POST /api/bids/ устанавливает блокировку строки таблицы auction через конструкцию SELECT FOR UPDATE [23], проверяет статус и сумму, вставляет строку в таблицу bids, обновляет поле current_price. Все операции выполняются в одной транзакции. Параллельные запросы сериализуются на уровне строки. Пропускная способность для разных лотов при этом не снижается.

3.4.2. Завершение торгов по событию

Модуль `app/services/auction_scheduler.py` удерживает две сопрограммы на каждый активный лот: «скоро завершится» (за пять минут до окончания) и «завершить» (вызов функции `complete_auction`). Хук жизненного цикла приложения `lifespan` пересоздаёт таймеры при перезапуске сервера через функцию `schedule_active_auctions()`.

Жизненный цикл таймеров задан как контракт. Продление дедлайна - ставка в окне антиснайпинга или ручное расширение через PATCH - вызывает `schedule_auction` повторно. Вспомогательная функция `_arm_completion_task` отменяет ранее занимавшую ячейку реестра задачу перед записью новой. Исключение - случай, когда ячейку занимала сама вызывающая корутина: попытка отменить себя выбросила бы `CancelledError` в окружающий `session.close()` и оставила бы соединение пула `asyncpg` с незакрытой транзакцией `FOR UPDATE`. Внутренний путь повторного вооружения внутри `_wait_and_complete` этот шаг пропускает.

Откат сессии в обработчике обернут в `asyncio.shield`. Сигнал `shutdown_scheduler`, поступивший во время отката, не прерывает операцию: без щита откат завершился бы наполовину, соединение пула вернулось бы в неопределённом состоянии, и следующая попытка получить соединение приводила бы к ошибке «another operation in progress» на стороне `asyncpg`. Контракт зафиксирован в модульной строке документации `auction_scheduler.py` (раздел о безопасности отмены задач).

3.4.3. Выбор ведущего планировщика через рекомендательную блокировку

При запуске `uvicorn` с `N` рабочими процессами (опция `--workers N`) стартует `N` независимых циклов событий. Каждый из них создал бы свои таймеры. Решение – планировщик выполняется ровно в одном процессе. На старте каждый рабочий процесс пытается взять `pg_try_advisory_lock(0x4C4F54555353434820)`

[21]. Получивший её процесс запускает планировщик; остальные обслуживают только HTTP- и WebSocket-трафик. Реализация функции `try_become_scheduler_leader` приведена в листинге Г.1 приложения Г.

3.4.4. Очередь исходящих писем с экспоненциальной задержкой повторов

Шаблон транзакционной очереди исходящих сообщений (outbox) [22] реализован в двух местах. Запись в очередь - функция `enqueue_email` - принимает необязательный параметр сессии SQLAlchemy. Если вызывающий код держит открытую сессию (регистрация пользователя, сброс пароля, рассылка уведомлений после завершения торгов), строка outbox вступает в ту же транзакцию, что и изменение основной таблицы. Один `commit` фиксирует обе записи атомарно. Сбой PostgreSQL посередине откатывает обе записи одновременно: состояния «пользователь создан, письмо потеряно» или «письмо в очереди, пользователя нет» не возникает. Без сессии (фоновый код вне HTTP-контекста) `enqueue_email` открывает собственный `SessionLocal` с обычным `commit`.

Фоновый обработчик `app/services/email_outbox.py` раз в 30 секунд читает партию готовых строк (FIFO по `created_at`, без блокировки) и захватывает их построчно через `SELECT ... FOR UPDATE SKIP LOCKED` [23]. При успехе устанавливается статус 'sent'. При ошибке инкрементируется счётчик `attempts`, поле `next_attempt_at` сдвигается по шкале 1/5/15/60/360 минут. После пяти неудачных попыток сообщение получает статус 'failed' и переходит в очередь отказавших - для последующего разбора оператором. Реализация выборки партии писем приведена в листинге Г.2 приложения Г.

3.4.5. Транзакционная целостность баланса

Все денежные операции – `buy_now`, `complete_auction`, `deposit`, `withdraw` – выполняют конструкцию `SELECT FOR UPDATE` на двух строках: `users` и `auctions`. Вспомогательная функция `lock_users_by_id` сортирует идентификаторы

перед захватом – упорядоченный захват предотвращает взаимные блокировки. На каждое движение средств записывается строка в таблицу transactions с полем balance_after – журнал движения средств поддается восстановлению. Реализация обработчика ставки с конкурентной защитой через SELECT FOR UPDATE приведена в листинге Г.3 приложения Г.

Денежные поля в схемах Pydantic объявлены как Decimal с decimal_places=2. Сумма с тремя знаками после запятой (например, 100.005) отклоняется на границе ответом 422 - без неявного округления внутри обработчика. Внутренние операции, которые правомерно дают дробные копейки (вычисление платформенной комиссии как процента от суммы продажи), приведены к единому округлению через утилиту quantize_money в app/utils/money.py. Политика округления (HALF_UP, два знака) сосредоточена в одном модуле.

3.4.6. Идемпотентность повторных уведомлений

Предупреждение «аукцион скоро завершится» фиксирует факт отправки флагом ending_soon_notified в строке лота. Прежняя реализация ставила флаг до начала рассылки. Сбой посреди цикла - недоступность одного из адресатов, кратковременный обрыв соединения с PostgreSQL - оставлял флаг True при том, что половина участников торгов писем не получала. Следующий тик планировщика выполнял ранний возврат по флагу и оставшимся адресатам письма не доставлял.

Фиксация флага перенесена за окончание рассылки. Каждый адресат предварительно проверяется на наличие строки Notification типа AUCTION_ENDING для текущего лота - при её наличии итерация пропускается. Сбой посреди цикла теперь оставляет флаг False; следующий тик повторяет рассылку и дозаполняет адресатов, которым доставка не удалась. Повторная отправка исключена за счёт проверки уже существующей строки. Гарантия доставки повышена с «не более одного раза» (at-most-once, с потерей на сбое) до «хотя бы один раз» (at-least-once) с фактической однократностью на адресата.

3.5. Реализация REST-интерфейса

49 ресурсов REST-интерфейса разнесены по 13 файлам в каталоге `app/routers/`. Шаблон обработчика: декларативная сигнатура `async def` с механизмом `Depends` объявляет параметры URL и тела запроса, внедряет провайдеры зависимостей (текущий пользователь, сессия базы данных, ограничитель частоты запросов). Тело функции содержит только бизнес-логику. SQL-запросы выстроены по классическим рекомендациям [25, 26] – явные JOIN-конструкции, селективные предикаты `WHERE`, индексированные столбцы в сортировке.

3.6. Реализация подсистемы реального времени

Менеджер `ConnectionManager` хранит два реестра: `active_connections` (отображение `auction_id` в множество сокетов) и `user_connections` (отображение `user_id` в множество сокетов). Рассылка – веерная по списку подписчиков; неактивные сокет удаляются при первой ошибке вызова `send_json`. Аутентификация для канала уведомлений выполняется через заголовок `Sec-WebSocket-Protocol` по RFC 6455 [19], а не через параметры строки запроса. Реализация функции широковещания с очисткой мёртвых соединений приведена в листинге Г.5 приложения Г.

3.7. Реализация подсистемы уведомлений

Тела электронных писем собираются из строковых шаблонов Python (f-строк) со встроенными стилями оформления – веб-интерфейс Gmail и настольный клиент Outlook корректно отображают оформление. Каждое подставляемое в письмо пользовательское поле экранируется функцией `html.escape` – защита от внедрения разметки (XSS) в почтовом клиенте получателя. Каждое письмо проходит через транзакционную очередь `outbox` перед отправкой по протоколу SMTP.

Веерная рассылка уведомлений нескольким адресатам - завершение торгов на двадцать участников торгов, рассылка NEW_LOT подписчикам активного продавца - выполняется через функцию `notify_many`. Прежняя реализация выполнялась последовательно: `notify_user` в цикле, каждая итерация - отдельный `commit` на сессии запроса плюс новое соединение пула под `INSERT` в `outbox`. Двадцать участников - сорок последовательных обращений к базе данных. Латентность планировщика на конец минуты росла линейно от числа участников лота.

Новый помощник построен по-иному. Все строки `Notification` - один `add_all` и один `commit`. Все строки `outbox` в той же транзакции (см. 3.4.4 об атомарной записи в очередь). `WebSocket-push` каждому получателю - `asyncio.gather` с `return_exceptions`, чтобы медленный клиент не блокировал остальных. Одно обращение к базе данных и параллельная отправка кадров. Латентность рассылки перестаёт расти линейно от числа адресатов; ограничивающим фактором становится самый медленный канал, а не их сумма.

3.8. Реализация клиентской части

Общая клиентская библиотека `static/js/common.js` предоставляет константы `window.API` и `window.WS_BASE`, функцию `esc` для защиты от межсайтовых скриптовых атак (XSS), форматтеры `fmtMoney` и `fmtDate`, обёртку `apiFetch` для вызова REST-интерфейса, функцию `showToast` для всплывающих уведомлений, а также автоматическую инициализацию счётчика уведомлений в верхней панели. Таблицы стилей CSS и сценарии JavaScript для каждой страницы подключаются индивидуально.

3.9. Развёртывание программного обеспечения

Локальная разработка – конфигурация `docker-compose.yml` запускает PostgreSQL 16 на порту 5433. Приложение запускается через `uvicorn` непосредственно. Для промышленного развёртывания достаточно сервисного юнита `systemd` с обратным прокси `nginx`.

3.10. Применение принципов качественного кода

Соблюдены принципы SOLID, DRY, KISS [12]. Принцип единственной ответственности (Single Responsibility) – каждый модуль в каталоге `app/services/` отвечает за одну подсистему: `auction_scheduler` – таймеры, `email_outbox` – рассылка писем, `scheduler_election` – координация. Принцип инверсии зависимостей (Dependency Inversion) – модуль `scheduler` не знает о `services.auctions`; обработчики завершения регистрируются через функцию `register_handlers` извне. Принцип отказа от дублирования (DRY) – общие операции вынесены во вспомогательные функции (`lock_users_by_id`, `fetch_auction_bidders`, `_seller_commission`). Принцип простоты (KISS) – отказ от брокера задач Celery [13], шины сообщений Redis pub/sub и микросервисной декомпозиции в пользу штатных примитивов Python [27] и PostgreSQL [11].

Из канонического каталога порождающих, структурных и поведенческих шаблонов проектирования [41] в коде явно прослеживаются четыре. Шаблон «Наблюдатель» (Observer) реализован классом `ConnectionManager` в модуле `app/services/websocket_manager.py`: сокеты подписчиков группируются по идентификатору аукциона и идентификатору пользователя, методы `broadcast` и `send_notification` – публикация события всем сокетам соответствующей группы с попутной отбраковкой мёртвых соединений. Шаблон «Шаблонный метод» (Template Method) – внутренний метод `_fan_out`, общий каркас обеих публикаций: снимок группы под защитой от мутаций на `await`, перебор сокетов с `send_json`, очистка реестра от мёртвых подключений; `broadcast` и `send_notification` отличаются только выбором реестра-аргумента. Шаблон «Одиночка» (Singleton) – единственный экземпляр `manager` создаётся на уровне модуля `websocket_manager.py` и переиспользуется всеми маршрутами в пределах процесса. Шаблон «Стратегия» (Strategy) – контекст хеширования паролей `passlib.CryptContext` со схемами `argon2` и `bcrypt` и параметром `deprecated="auto"` в файле `app/utils/security.py`: алгоритм проверки выбирается по префиксу сохранённого хеша, унаследованные `bcrypt`-хеши прозрачно ротируются в

argon2id при первой успешной верификации пароля. Транзакционная очередь исходящих сообщений (раздел 3.4.4) и схема «производитель – потребитель» (раздел 3.7) относятся к шаблонам корпоративной интеграции [12], выходящим за пределы канонических двадцати трёх шаблонов GoF.

Выводы по главе 3

Реализация повторяет проектные решения главы 2. Реализованы 49 ресурсов REST-интерфейса, два WebSocket-канала, восемь фоновых задач. Транзакционная целостность достигнута через конструкцию SELECT FOR UPDATE с упорядоченным захватом строк для предотвращения взаимных блокировок. Координация многопроцессного развёртывания – рекомендательная блокировка PostgreSQL. Доставка электронной почты – очередь outbox с экспоненциальной задержкой повторов. Клиентская часть – многостраничное приложение с обновлениями через WebSocket и экспоненциальной задержкой повторного подключения.

Граница транзакционной долговечности в подсистеме уведомлений сведена в одну точку: INSERT в outbox разделяет сессию с изменением основной таблицы и фиксируется одним commit. Веерная рассылка на множество адресатов выполнена пакетной вставкой и параллельной отправкой по каналам - линейная зависимость латентности от числа участников торгов устранена. Отмена задач планировщика описана как контракт: щит asyncio.shield на откате, отказ от самоотмены, перенос очистки реестра в блок finally с проверкой принадлежности задачи. Идемпотентность повторных уведомлений обеспечена проверкой уже существующих строк Notification, что переводит контракт доставки из «не более одного раза» в «хотя бы один раз».

ГЛАВА 4. ТЕСТИРОВАНИЕ И ОБЕСПЕЧЕНИЕ КАЧЕСТВА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

4.1. Стратегия тестирования

Тестирование организовано по трём уровням: модульное (изолированные функции и классы), интеграционное (несколько модулей с реальной БД), системное (сквозные сценарии через HTTP). Применены `pytest` 9 и `pytest-asyncio` [2]. Тестовый контейнер PostgreSQL 16 разворачивается через `docker-compose` [28]. Для ускорения в `conftest.py` вместо Alembic используется `Base.metadata.create_all` – таблицы создаются за миллисекунды.

4.2. Модульное и интеграционное тестирование

291 автоматизированный тестовый сценарий в 25 файлах. Все 49 ресурсов REST-интерфейса покрыты сценарием штатного использования и минимум одним негативным сценарием. Оба WebSocket-маршрута проверены прямым вызовом обработчика с подменённым сокетом: контракт лимита соединений с одного IP, скользящее окно ограничения частоты сообщений, ping по таймауту приёма, отклонение по `subprotocol`-токену, проверка соответствия `token_version` – все ветви аутентификации и обработки ошибок включены в регрессионный набор.

4.2.1. Тестирование конкурентных операций

Десять сценариев имитируют состояния гонки через `asyncio.gather`. Например: 20 параллельных ставок на один лот через `gather` – проходит ровно одна. Без таких тестов гонка проявилась бы только в промышленной эксплуатации на пиковой нагрузке.

4.2.2. Покрытие кода тестами

Запуск `pytest --cov=app` даёт численное покрытие по модулям. Конкурентный режим coverage с дополнением `thread+greenlet` (см. `pyproject.toml`)

нужен из-за специфики `httpx.ASGITransport` – без него ~37% `async`-кода не учитывается.

Таблица 4.1 – Покрытие кода по подсистемам (pytest-cov)

Подпакет	Statements	Missed	Coverage, %
app/routers/	1013	58	94
app/services/	650	107	84
app/models/	190	0	100
app/schemas/	209	6	97
app/utils/	201	21	90
Итого	2263	192	91

4.3. Нагрузочное тестирование

Для количественной оценки выбран программный интерфейс каталога аукционов – ресурс, к которому обращается каждый посетитель сайта при первой загрузке страницы. Сценарий нагрузки идентичен для сравниваемых стеков: 2000 запросов GET к ресурсу `/api/auctions`, сто одновременных подключений, фильтр по статусу `active`, страница из двенадцати объектов. Сценарий реализован скриптом `benchmarks/load_test.py`, использующим общий клиент `httpx.AsyncClient`. Перед измерением выполнено пять разогревочных запросов – чтобы исключить из выборки задержку установления TCP-соединения.

4.3.1. Конфигурация испытательного стенда

Измерения выполнены на машине разработчика под управлением операционной системы Windows 10, версия интерпретатора Python 3.12. Система управления базами данных PostgreSQL 16-alpine развёрнута в контейнере Docker на порту 5433. Пул соединений SQLAlchemy одинаков для обоих сравниваемых стеков: `pool_size = 10`, `max_overflow = 20`. Асинхронный сервер – `uvicorn` без дополнительных рабочих процессов (один процесс). Синхронный сервер – `waitress` с числом рабочих потоков 1, 4 и 8. Синхронный обработчик размещён в файле `benchmarks/sync_baseline.py`: используется та же ORM-модель, идентичные SQL-запросы, отличие – синхронный драйвер `psycopg2` вместо асинхронного `asyncpg`.

Таблица 4.2 – Результаты нагрузочного тестирования (сто одновременных подключений, 2000 запросов)

Конфигурация	Запросов/с	p50, мс	p95, мс	p99, мс	Ошибок, %
uvicorn, ограничитель частоты включён	72,7	853	4300	6500	0
uvicorn, ограничитель частоты выключен	77,3	901	3500	5700	0
uvicorn, без Pydantic-валидации ответа	99,4	637	2800	5000	0
waitress, один поток	65,4	680	801	982	0,1
waitress, четыре потока	96,0	723	3200	5000	0
waitress, восемь потоков	98,7	718	2900	4900	0

4.3.2. Анализ выявленного узкого места

Результаты на первый взгляд противоречат архитектурным ожиданиям: синхронный сервер с четырьмя потоками превосходит асинхронный по пропускной способности почти на четверть. Причина не в архитектуре, а в ограничениях одного процесса Python под глобальной блокировкой интерпретатора (Global Interpreter Lock, GIL).

Асинхронный обработчик затрачивает порядка 13 миллисекунд процессорного времени на каждый запрос: десериализация параметров запроса, валидация ответа средствами библиотеки Pydantic, обход графа связанных объектов (опция selectinload в SQLAlchemy), формирование JSON-документа. На время выполнения этих операций цикл событий не переключается между сопрограммами; библиотека `asynpg` обслуживает сетевые операции в промежутках. Расчётная оценка предельной пропускной способности $1000 / 13 \approx 77$ запросов в секунду совпадает с результатом измерения.

Синхронный сервер `waitress` с четырьмя потоками опережает асинхронный по другой причине. Драйвер `psycopg2` во время ожидания ответа от СУБД

освобождает глобальную блокировку интерпретатора – четыре потока физически параллельно ожидают результата запроса к PostgreSQL. Прирост пропускной способности между четырьмя и восемью потоками составляет лишь 2,8 запроса в секунду: ограничение возникает не на уровне числа потоков, а на уровне планировщика СУБД и общего пула соединений.

4.3.3. Эксперимент: стоимость валидации ответа средствами Pydantic

Выполнен отдельный замер с временным отключением валидации ответа на асинхронном маршруте (параметр `response_model` декоратора маршрута). Пропускная способность увеличилась с 77 до 99 запросов в секунду; медианная задержка снизилась с 901 до 637 миллисекунд. Валидация ответа обходится приблизительно в 28% пропускной способности на рассматриваемом маршруте. В промышленной эксплуатации контракт программного интерфейса имеет приоритет над пропускной способностью – валидация сохранена в производственном коде. Замер фиксирует резерв оптимизации: переход на сериализатор `orjson` через класс `ORJSONResponse` либо отложенная валидация, выполняемая только в режиме тестирования.

4.3.4. Проекция результатов на промышленную конфигурацию

Главный вывод состоит не в превосходстве синхронной модели, а в том, что измерение на одном процессе ограничено глобальной блокировкой интерпретатора до того, как цикл событий получает возможность раскрыть конкурентность. В промышленной конфигурации `gunicorn` с четырьмя рабочими процессами (опция `-w 4`) и классом `UvicornWorker` четыре независимых цикла событий обслуживают по сто одновременных HTTP-сессий каждый – суммарно порядка 400 активных клиентов программного интерфейса и одновременно тысячи открытых WebSocket-соединений на один узел развёртывания. Для синхронного стека той же конфигурации потребовалось бы $4 \times 100 = 400$ потоков или процессов – модель не масштабируется ни по памяти, ни по числу подключений к PostgreSQL.

Узкое место в однопроцессном измерении на машине разработчика – не сеть и не СУБД, а процессорные затраты на сериализацию ответа. В реальной нагрузке с массовой рассылкой ставок через WebSocket-соединения, исходящими вызовами к платёжному провайдеру, пакетной отправкой писем из транзакционной очереди исходящих сообщений доля ожидания ввода-вывода возрастает, доля процессорной работы снижается. Эта пропорция и оправдывает архитектурный выбор асинхронной модели.

4.3.5. Микрозамер веерной рассылки уведомлений

Сценарий завершения аукциона рассылает уведомление каждому участнику торгов: запись в таблицу Notification, постановка письма в очередь EmailOutbox, рассылка по открытым WebSocket-соединениям. Прежняя реализация обрабатывала участников последовательно – по одному вызову `notify_user` на адресата. Каждая итерация фиксировала транзакцию для записи уведомления через сессию вызывающего кода, после чего вспомогательная функция `_fire_and_forget_email` открывала собственную сессию `SessionLocal` и отдельным `commit` сохраняла строку в очередь исходящих сообщений. Число обращений к PostgreSQL росло линейно с числом участников – по две фиксации транзакции на адресата. Введённый в разделе 3.7 пакетный помощник `notify_many` объединяет записи в два множественных INSERT (по одному на таблицу Notification и EmailOutbox) под общей транзакцией, а отправку по WebSocket-соединениям запускает через `asyncio.gather` после фиксации.

Замер выполнен скриптом `benchmarks/notify_fanout.py`: PostgreSQL 16 в контейнере Docker, изолированная тестовая база, обработчик WebSocket-соединений заменён заглушкой – архитектурное решение касается персистентной стороны, сетевой канал не влияет на сравнение. Для варианта с последовательным циклом историческое поведение `_fire_and_forget_email` восстановлено через `monkeypatch` – без него прямой прогон текущего кода в цикле уже использует общую сессию и недопустимо занижает стоимость "до". Оба пути обращаются к одному предварительно созданному аукциону, поэтому

каждая запись Notification несёт настоящий внешний ключ и проходит ту же проверку foreign-key, что и в продакшене. По 12 измеренных итераций на каждое значение размера веерной рассылки, 2 прогрева отброшены. Между итерациями таблицы Notification и EmailOutbox очищаются: оба пути начинают с одинакового исходного состояния.

Таблица 4.3 – Сравнение пакетной и последовательной веерной рассылки уведомлений

Получателей	Цикл, мс	Пакет, мс	Ускорение	Коммитов цикл / пакет
5	850	162	5,2x	10 / 1
10	1604	169	9,5x	20 / 1
20	2988	181	16,6x	40 / 1
50	7685	250	31x	100 / 1
100	14 563	364	40x	200 / 1

Кривая последовательного пути растёт линейно – около 144 миллисекунд на каждого получателя. Эта величина складывается из коммита транзакции с записью Notification на стороне вызывающего кода и независимого пути SessionLocal → INSERT → commit для одной строки EmailOutbox. Пакетный путь практически не зависит от размера рассылки. Средняя стоимость одной добавочной строки внутри множественных INSERT сохраняется на уровне двух миллисекунд даже при 100 адресатах: фиксированная стоимость одного вызова notify_many (открытие сессии, два множественных INSERT, общий commit, refresh первичных ключей) амортизируется по всем получателям. Пакетный путь выигрывает уже с одного участника – накладные расходы цикла с восстановленным историческим швом доминируют над работой Python даже на минимальной аудитории. Опорный сценарий из задания – аукцион с двадцатью участниками – завершается за 181 миллисекунду вместо трёх секунд.

Выигрыш не сводится к удобному API. Под нагрузкой сокращение числа обращений к СУБД с 2N до двух освобождает соединения пула под приём ставок и обслуживание регистраций. Всплеск одновременных завершений (большой пакет лотов с близкими дедлайнами) перестаёт быть угрозой для пропускной способности всего сервиса. Тот же помощник переиспользован при создании

лота в маршруте POST /api/auctions для верной рассылки подписчикам продавца.

4.4. Анализ угроз и тестирование безопасности

Реализованная защита сопоставлена с OWASP Top 10 (2021) и стандартом OWASP ASVS v4.0.3 [29]. Хеширование паролей выполнено по Argon2id согласно RFC 9106 [30]. Таблица 4.4 фиксирует каждую угрозу – реализованную меру – место в коде. Тесты безопасности размещены в tests/test_auth.py, test_password_reset.py, test_rate_limit.py.

Таблица 4.4 – Соответствие защиты OWASP Top 10

Угроза	Реализованная защита	Где
A01 Broken Access Control	JWT с проверкой token_version, claim'ами iss/aud + per-endpoint owner-check	utils/security.py, routers/auctions.py
A02 Cryptographic Failures	Argon2id (OWASP-рекомендованный), bcrypt-legacy с прозрачным rehash	utils/security.py
A03 Injection	SQLAlchemy ORM + параметризованные запросы; ограничение charset логина (латиница/кириллица/_-) защищает поток имени пользователя в шаблоны от stored-XSS	models/, services/, schemas/user.py
A04 Insecure Design	FOR UPDATE при денежных операциях, упорядоченный захват идентификаторов для предотвращения взаимных блокировок	services/balance.py
A05 Security Misconfiguration	X-Frame-Options DENY, X-Content-Type-Options nosniff, Content-Security-Policy, Referrer-Policy, Permissions-Policy, CORS-allowlist; dev Postgres биндится только на 127.0.0.1	app_factory.py, docker-compose.yml
A06 Vulnerable Components	версии зависимостей зафиксированы; pip-audit на полном дереве requirements.txt запускается отдельным job в CI на	.github/workflows/ci.yml, requirements.txt

	каждый PR (сверка с базами PyPI Advisory и OSV)	
A07 Authentication Failures	slowapi 10/min на /login по IP + per-account exponential lockout (5/10/15/20 неудач → 1м/5м/15м/1ч); timing-stable verify через dummy hash; массовая инвалидация через token_version с FOR UPDATE на смене пароля; access-токен действует 2 часа	routers/auth.py, utils/security.py
A08 Software/Data Integrity	Alembic-миграции в CI до тестов, immutable transactions journal, CI-gate покрытия --cov-fail-under=80	.github/workflows/ci.yml, models/transaction.py
A09 Security Logging	JSON-logging по уровню/логгеру; LOG_FORMAT=json для shipper'ов	app_factory.py
A10 SSRF	Валидация URL изображений: отвергаются javascript: и data: схемы; rolling-24h byte-quota на загрузки (50 МБ/сутки на пользователя) ограничивает злоупотребление /upload-image как бесплатным хостингом	utils/images.py, routers/uploads.py

4.5. Найденные дефекты и анализ качества

В ходе разработки выявлено и устранено шесть дефектов, представляющих различную степень критичности. Каждый из них имеет воспроизводимый автоматизированный тестовый сценарий – регрессия закрыта на уровне исполняемых проверок системы непрерывной интеграции.

Таблица 4.5 – Найденные и устранённые дефекты

№	Критичность	Описание	Где исправлено	Регресс-тест
1	Критический	Состояние гонки при одновременных вызовах /buy-now приводило к двойному списанию	services/auctions.py – конструкция SELECT FOR UPDATE на строке аукциона	test_balance.py::test_concurrent_buy_now_only_one_succeeds

		средств с покупателя		
2	Критический	Состояние гонки при размещении и ставок на разные лоты позволяло одному пользователю перерасходовать баланс: каждая ставка независимо проходила проверку доступных средств	routers/bids.py + services/balance.py – блокировка строки пользователя через lock_users_by_id с упорядоченным захватом для предотвращения взаимных блокировок	test_bids.py::test_concurrent_cross_auction_bids_respect_balance
3	Критический	Потеря копеек при арифметике баланса в типе данных float: накопление округления при длинных цепочках операций	Миграция базы данных с SQLite на PostgreSQL с типом NUMERIC(12,2); все денежные значения проходят через утилиту app/utils/money.py::to_decimal	test_balance.py::test_concurrent_deposits_and_apply
4	Значительный	Токен сброса пароля валидировался без инкремента поля token_version: ссылка из письма работала повторно после успешного сброса	routers/auth.py – блокировка строки пользователя с проверкой token_version в той же транзакции	test_password_reset.py::test_token_single_use

5	Значительный	Утечка дескрипторов WebSocket-сокетов: мёртвые соединения оставались в реестре менеджера, реестр рос неограниченно при цикле переподключений клиента	services/websocket_manager.py – удаление сокета при первой ошибке вызова send_json	test_websocket_manager.py::test_broadcast_drops_dead_connections
6	Значительный	Лот с фиксированной ценой (auction_type=bin) принимал ставки, что позволяло поднять current_price выше bin_price, а другому пользователю выкупить лот через buy-now по более низкой цене	routers/bids.py – явный отказ от обработки ставок на BIN-лотах	test_bids.py::test_bid_on_bin_lot_rejected

4.6. Непрерывная интеграция и валидация развёртывания

.github/workflows/ci.yml запускает ruff lint + pip-audit + alembic upgrade head + pytest -v на каждый PR в main и develop. Отдельный job pip-audit сверяет полное дерево зависимостей requirements.txt с базами PyPI Advisory и OSV (см. раздел 4.4, строка A06). Сервис postgres:16-alpine поднимается в job через healthcheck. Шаг alembic upgrade head обнаруживает рассинхронизированные миграции до запуска тестов.

Выводы по главе 4

Тестирование выполнено комплексно: 291 автоматизированный сценарий (включая 10 проверок конкурентного доступа через `asyncio.gather`); нагрузочное профилирование шести конфигураций программного стека выявило ограничение по процессорной нагрузке на одном процессе при ста одновременных подключениях и зафиксировало резерв порядка 28% при переходе на класс `ORJSONResponse`; защита по списку OWASP Top 10 покрыта на уровне реализации и тестов. Микрозамер веерной рассылки уведомлений подтвердил эффект архитектурного решения о пакетировании: завершение аукциона с двадцатью участниками занимает 181 миллисекунду против трёх секунд у исторической последовательной реализации – ускорение в 16,6 раза, а на сотне получателей разрыв достигает сорока раз. Система непрерывной интеграции прогоняет полный цикл (статический анализ, миграции базы данных, тестовый прогон) на каждый запрос на внесение изменений. Продукт удовлетворяет нефункциональным требованиям, сформулированным в первой главе.

ГЛАВА 5. ТЕХНИКО-ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ

5.1. Описание программного продукта и цели разработки

Объект ТЭО – веб-сервис «Лотус». Бизнес-модель – комиссия 7% на стороне продавца. Цель – занять освободившуюся после ухода eBay нишу C2C-аукционов на российском рынке.

5.2. Позиционирование на рынке

Сравнение с Avito Аукционы, Au.ru и «Мешок» приведено в разделе 1.2. Сильные стороны «Лотуса» – real-time по WebSocket, OpenAPI, обе модели торгов (BID+BIN), открытая технологическая база. Слабые стороны MVP – отсутствие узнаваемой торговой марки и мобильного приложения – учтены в комиссионной ставке.

5.3. Планирование разработки

Календарный план – восемь этапов жизненного цикла, последовательно с частичным наложением фаз тестирования и документирования на этап реализации. Производительность одного разработчика – 160 рабочих часов в месяц.

Таблица 5.1 – Трудоёмкость и календарный план разработки

№	Этап	Трудоёмкость, чел.-час	Срок, мес.
1	Анализ предметной области, выявление требований	80	0,5
2	Проектирование архитектуры и схемы БД	100	0,6
3	Реализация модели данных и миграций	60	0,4
4	Реализация бизнес-логики и фоновых подсистем	240	1,5
5	Реализация REST API и WebSocket	120	0,8

6	Реализация клиентской части	120	0,8
7	Тестирование и нагрузочное профилирование	100	0,6
8	Документирование и подготовка к развёртыванию	60	0,4
	Итого	880	5,5

5.4. Расчёт себестоимости разработки

5.4.1. Заработная плата основная

Оклад разработчика начального уровня в Ставрополе (2026 г.) – 45 000 руб./мес. Срок 5,5 месяца. $ЗП_{осн} = 45\,000 \times 5,5 = 247\,500$ руб.

5.4.2. Заработная плата дополнительная

12% от основной – оплата отпуска и больничного. $ЗП_{доп} = 29\,700$ руб.
 $ФОТ = 277\,200$ руб.

5.4.3. Отчисления во внебюджетные фонды

Базовая ставка ОСН – 30% от ФОТ = 83 160 руб. Льготная ставка для аккредитованных ИТ-организаций (Постановление Правительства РФ № 1729 от 30.09.2022 [31]) – 7,6% от ФОТ = 21 067 руб.

5.4.4. Материальные затраты и амортизация оборудования

$МЗ = 3\,000 \times 5,5 = 16\,500$ руб. Амортизация ноутбука (стоимость 150 000 руб., линейный метод с периодом полезного использования 36 месяцев) за 5,5 месяца = 22 917 руб.

5.4.5. Накладные расходы

16% от прямых затрат. Полная себестоимость с ИТ-льготой – 391 713 руб., без льготы – 463 741 руб.

Таблица 5.2 – Структура себестоимости разработки, руб.

Статья	Без IT-льготы	С IT-льготой	Доля, %
Заработная плата основная	247 500	247 500	63,2
Заработная плата дополнительная (12%)	29 700	29 700	7,6
Страховые взносы (30% / 7,6%)	83 160	21 067	5,4
Материальные затраты	16 500	16 500	4,2
Амортизация оборудования	22 917	22 917	5,9
Накладные расходы (16% от прямых)	63 964	54 029	13,8
Полная себестоимость	463 741	391 713	100,0

5.5. Расчёт цены и прогноз выручки

Комиссия 7% от каждой завершённой сделки. Постепенный выход на плановую аудиторию: месяц 1 – 10 активных продавцов; месяц 6 – 50 продавцов; месяц 12 – 100 продавцов.

Таблица 5.3 – Прогноз выручки по месяцам после запуска MVP

Месяц	Продавцов	Оборот/продавец, руб.	Общий оборот, руб.	Выручка (7%), руб.
1	10	5 000	50 000	3 500
3	20	10 000	200 000	14 000
6	50	20 000	1 000 000	70 000
9	75	35 000	2 625 000	183 750
12	100	50 000	5 000 000	350 000

5.6. Оценка экономии за счёт асинхронной архитектуры

Архитектурное преимущество асинхронного стека проявляется не на одиночных запросах REST-интерфейса: пропускная способность двух стеков на одном процессе сопоставима (см. раздел 4.3) – 77 запросов в секунду у uvicorn против 65 запросов в секунду у waitress с одним потоком. Преимущество раскрывается в характеристике одновременных WebSocket-соединений. К завершению торгов один лот собирает до тысячи зрителей, подписанных на трансляцию обновлений ставок в реальном времени. Каждая такая сессия удерживается часами.

Один процесс `uvicorn` удерживает подобные сессии без выделения отдельного потока операционной системы на каждое соединение: оперативная память расходуется только на сетевой буфер и контекст сопрограмы – порядка 30–60 КБ на одно соединение. Синхронный стек на серверах `waitress` или классическом `gunicorn` требует отдельного потока (или процесса) на каждую сессию; стек одного потока в Python на платформе Linux по умолчанию занимает не менее 8 МБ оперативной памяти. Разница масштабируется по числу одновременных подключений и определяет минимальную конфигурацию виртуального сервера.

Таблица 5.4 – Сравнение инфраструктурных затрат на 5 лет при целевой нагрузке 1000 одновременных подписчиков WebSocket

Параметр	Асинхронный стек (<code>uvicorn</code>)	Синхронный стек
Память на 1000 одновременных сессий WebSocket	около 60 МБ	около 8 ГБ (1000 потоков по 8 МБ)
Пропускная способность одного процесса на ресурсе <code>/api/auctions</code>	77 запр./с	65 запр./с (<code>waitress</code> , один поток)
Минимальная конфигурация виртуального сервера	1 vCPU, 1 ГБ ОЗУ	4 vCPU, 8 ГБ ОЗУ
Стоимость виртуального сервера, руб./мес.	500	4 000
Стоимость за 60 месяцев, руб.	30 000	240 000
Экономия за 5 лет, руб.	–	210 000

Численные значения по памяти получены прямым измерением: 1000×8 МБ = 8 ГБ для модели «поток на соединение» – фундаментальное ограничение, не зависящее от языка программирования или фреймворка. Асинхронный стек обходит его за счёт того, что сопрограмма не блокирует поток операционной системы – тысячи сопрограмм разделяют один поток.

5.7. Дисконтированные показатели экономической эффективности

Ставка дисконтирования $r = 20\%$ годовых (ключевая ставка ЦБ 16% + премия за риск 4 п.п. для IT-стартапа на стадии MVP). Первоначальные инвестиции $CF_0 = 391\,713$ руб. Эксплуатационные расходы – 600 000 руб./год.

Налог на прибыль = 0% по льготе для аккредитованных ИТ (ФЗ № 67-ФЗ от 26.03.2022, ст. 149 НК РФ [32]).

Таблица 5.5 – Денежные потоки и их дисконтирование

Год t	CF_t , руб.	Коэф. $1/(1+r)^t$	PV_t , руб.	Накопл. PV , руб.
0	-391 713	1,0000	-391 713	-391 713
1	900 000	0,8333	750 000	358 287
2	3 600 000	0,6944	2 500 000	2 858 287
3	3 600 000	0,5787	2 083 333	4 941 620
4	3 600 000	0,4823	1 736 111	6 677 731
5	3 600 000	0,4019	1 446 759	8 124 491

$NPV = 8\,124\,491$ руб. $PI = (NPV + CF_0) / |CF_0| = 21,7$. $IRR > 200\%$ годовых.

Простой срок окупаемости – 5,2 мес., дисконтированный – 6,3 мес.

Чувствительность: при $r = 30\%$ NPV остаётся положительным ($\approx 5,5$ млн).

5.8. Сводные технико-экономические показатели

Таблица 5.6 – Основные технико-экономические показатели проекта

Показатель	Значение
Трудоёмкость разработки, чел.-час	880
Срок разработки, мес.	5,5
Численность исполнителей, чел.	1
Себестоимость без льгот, руб.	463 741
Себестоимость с льготой 7,6%, руб.	391 713
Прогноз годовой выручки, руб.	4 200 000
ОРЕХ, руб./год	600 000
Чистая годовая прибыль, руб.	3 600 000
Ставка дисконтирования, % годовых	20
NPV (5 лет), руб.	8 124 491
PI	21,7
IRR , %	> 200
Простой срок окупаемости, мес.	5,2
Дисконтированный срок окупаемости, мес.	6,3
Экономия на инфраструктуре за 5 лет, руб.	210 000

Выводы по главе 5

Себестоимость разработки 391 713 руб. с ИТ-льготой. Прогноз чистой месячной прибыли на плато – 300 000 руб. NPV на 5 лет – 8,1 млн при $r = 20\%$; IRR превышает ставку дисконтирования в 10 раз; дисконтированный срок окупаемости – 6,3 месяца. Асинхронная архитектура даёт дополнительную экономию 210 000 руб. на инфраструктуре за пятилетний период – за счёт

удержания тысяч одновременных сессий WebSocket на одном процессе без выделения отдельного потока операционной системы на каждое соединение. Проект экономически целесообразен.

ГЛАВА 6. БЕЗОПАСНОСТЬ И ОХРАНА ТРУДА

6.1. Анализ опасных и вредных производственных факторов на рабочем месте программиста

Разработка «Лотуса» ведётся в условиях, типовых для IT-отрасли – офисное или домашнее рабочее место с персональным компьютером, длительная сидячая работа с высокой концентрацией на экранах мониторов. Классификация производственных факторов установлена ГОСТ 12.0.003-2015 [33]. Десять выделенных факторов сведены в таблице 6.1.

Таблица 6.1 – Опасные и вредные факторы на рабочем месте программиста

Группа	Фактор	Нормативный документ
Физические	Электромагнитное излучение от ПЭВМ	ГОСТ Р 50923-96
Физические	Недостаточная освещённость	СП 52.13330.2016
Физические	Повышенный шум	СанПиН 1.2.3685-21
Физические	Неблагоприятный микроклимат	СанПиН 1.2.3685-21
Физические	Опасность поражения током	ГОСТ 12.1.038-82
Психофизиологические	Монотонность труда	ГОСТ 12.0.003-2015
Психофизиологические	Статическая поза	ГОСТ 12.2.032-78
Психофизиологические	Зрительное напряжение	ГОСТ Р 50923-96
Социально-психологические	Эмоциональное напряжение	ГОСТ 12.0.003-2015
Физические	Опасность возгорания	СП 12.13130.2009

6.2. Мероприятия по обеспечению безопасности

6.2.1. Эргономические требования к рабочему месту

По ГОСТ 12.2.032-78 [34] рабочее место сидя должно обеспечивать: высоту сиденья 400–500 мм, регулировку подлокотников, поясничную опору. Высота стола – 680–800 мм. Расстояние от глаз до монитора – 500–700 мм согласно ГОСТ Р 50923-96 [35]. Угол наклона экрана – $\pm 15^\circ$ от вертикали.

6.2.2. Расчёт искусственного освещения

Метод коэффициента использования светового потока: $F = (E \times S \times Z \times K) / (n \times \eta)$, где $E = 300$ лк (СП 52.13330.2016 [36] для помещений с ПК), $S = 18$ м²,

$Z = 1,1$, $K = 1,5$, $\eta = 0,5$. Получаем $F = 17\,820$ лм. При LED-светильнике 3 600 лм необходимо 5 светильников.

6.2.3. Параметры микроклимата

Категория работ Ia (лёгкая работа сидя, энерготраты ≤ 139 Вт) согласно СанПиН 1.2.3685-21 [37]. Допустимые параметры – таблица 6.2.

Таблица 6.2 – Допустимые параметры микроклимата (СанПиН 1.2.3685-21)

Параметр	Холодный период	Тёплый период
Температура воздуха, °C	22–24	23–25
Относительная влажность, %	40–60	40–60
Скорость движения воздуха, м/с	$\leq 0,1$	$\leq 0,1$
Категория работ	Ia	Ia

6.3. Электро- и пожарная безопасность

6.3.1. Пожарная безопасность

Категория помещения по пожарной опасности – В4 (пониженная) по СП 12.13130.2009 [38]. Основные горючие материалы – бумага и пластиковые корпуса оборудования. Меры: огнетушители ОП-4 и ОУ-3 (1 комплект на 50 м²); автоматическая пожарная сигнализация с дымовыми извещателями ИП 212-141; противопожарные инструктажи каждые 6 месяцев.

6.3.2. Электробезопасность

По ГОСТ 12.1.038-82 [39] предельно допустимое напряжение прикосновения для длительной работы – 36 В переменного тока, 60 В постоянного. Помещение – без повышенной опасности (класс I). Меры: УЗО на 30 мА; заземление металлических корпусов; периодическая проверка изоляции.

6.3.3. Утилизация электронных отходов

По ФЗ № 89-ФЗ от 24.06.1998 [40] электронные отходы относятся к IV классу опасности. Сдача в специализированную лицензированную организацию.

Хранение до сдачи – в отдельно выделенной зоне, исключающей контакт с пищевыми продуктами.

Выводы по главе 6

Проведён анализ 10 вредных и опасных факторов рабочего места программиста. Применимые нормативы – ГОСТ 12.0.003-2015, СП 52.13330.2016, СанПиН 1.2.3685-21, ГОСТ 12.1.038-82, СП 12.13130.2009, ФЗ № 89-ФЗ. Категория помещения по пожарной опасности – В4. Меры безопасности позволяют поддерживать соответствие законодательным требованиям охраны труда.

ЗАКЛЮЧЕНИЕ

В рамках выпускной квалификационной работы решены пять задач, поставленных во введении. Разработан и протестирован веб-сервис «Лотус» – аукционная площадка с обновлением цен в режиме реального времени и обоснованным применением асинхронных технологий.

Проведён анализ предметной области: исследованы аналоги (Avito Аукционы, Au.ru, «Мешок», ушедший eBay), выделены три сегмента целевой аудитории, сформулированы функциональные и нефункциональные требования по IEEE 830-1998. Обоснован стек – FastAPI, PostgreSQL + asyncpg, WebSocket, asyncio.Task. Архитектура описана по C4 на трёх уровнях; разработана схема БД и контракт REST API; спроектированы четыре асинхронные подсистемы.

Реализована серверная часть на фреймворке FastAPI: 49 REST-обработчиков и два WebSocket-маршрута. Транзакционная целостность обеспечена конструкцией SELECT FOR UPDATE с упорядоченным захватом строк для предотвращения взаимных блокировок. Клиентская часть – многостраничное HTML-приложение с общими утилитами и WebSocket-клиентом с экспоненциальной задержкой повторного подключения. Координация в многопроцессном развёртывании опирается на три примитива: выбор ведущего узла через вызов pg_try_advisory_lock; очередь писем по шаблону транзакционной очереди исходящих сообщений (outbox) с конструкцией SELECT FOR UPDATE SKIP LOCKED; завершение торгов по событию через сопрограмму asyncio.Task с восстановлением состояния после перезапуска.

Тестирование проведено комплексно: 291 автоматизированный сценарий в 25 файлах, включая 10 проверок конкурентного доступа через asyncio.gather. Нагрузочное профилирование шести конфигураций программного стека выявило ограничение по процессорной нагрузке в однопроцессном режиме и зафиксировало резерв порядка 28% при отключении валидации ответа средствами библиотеки Pydantic; архитектурное преимущество асинхронной

модели проявляется в характеристике одновременных WebSocket-соединений, а не на одиночных запросах REST-интерфейса.

Технико-экономическое обоснование подтверждает целесообразность проекта: себестоимость разработки 391 713 руб. с применением IT-льготы 7,6%; чистая приведённая стоимость на горизонте пяти лет составила 8,1 млн руб. при ставке дисконтирования 20% годовых; дисконтированный срок окупаемости – 6,3 месяца.

Шестая глава фиксирует требования безопасности труда на рабочем месте разработчика и соответствие действующим нормативам охраны труда.

Пять архитектурных решений имеют ценность за пределами аукционной тематики:

- завершение задач по событию на сопрограмме `asyncio.Task` с восстановлением состояния после перезапуска – альтернатива опросу с фиксированным интервалом и планировщику `APScheduler` при секундной точности и отсутствии внешних зависимостей;
- выбор ведущего узла через рекомендательную блокировку PostgreSQL – система управления базами данных в роли координационного примитива без привлечения ZooKeeper, etcd или Redis;
- шаблон транзакционной очереди исходящих сообщений (outbox) с экспоненциальной задержкой повторов и очередью отказавших сообщений на конструкции `SELECT FOR UPDATE SKIP LOCKED` – для надёжной доставки сообщений во внешние системы;
- массовая инвалидация сессий через поле `token_version` одним SQL-запросом `UPDATE` – закрывает все JWT-сессии и WebSocket-соединения при смене или сбросе пароля;
- автоматическое продление дедлайна события при поступлении ввода в закрывающем окне – опирается на существующую инфраструктуру самовосстанавливающегося `asyncio`-таймера без правок планировщика, применимо к любому процессу с жёстким временным дедлайном (аукционы, тендерные торги).

Направления развития – автоматический переход резервного узла в ведущий с контролем по сигналам активности соединения; полнотекстовый поиск через расширение pg_trgm и индекс GIN; двухфакторная аутентификация по алгоритму TOTP; административные инструменты модерации; мобильное приложение; готовый к промышленному развёртыванию образ Docker с метриками системы мониторинга Prometheus и трассировкой через стандарт OpenTelemetry; сквозные тесты через библиотеку Playwright; интеграция с реальной платёжной системой. Цель выпускной квалификационной работы достигнута – веб-приложение разработано, протестировано и технико-экономически обосновано.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. IEEE Std 830-1998. IEEE Recommended Practice for Software Requirements Specifications [Текст]. – New York : IEEE, 1998. – 37 p.
2. pytest Documentation : Helps You Write Better Programs [Электронный ресурс] / Holger Krekel et al. – Режим доступа: <https://docs.pytest.org> (дата обращения: 12.05.2026).
3. Hattingh, C. Using Asyncio in Python: Understanding Python's Asynchronous Programming Features [Текст] / Caleb Hattingh. – Sebastopol : O'Reilly Media, 2020. – 166 p.
4. Ramalho, L. Concurrency Models in Python : Async vs Threads vs Multiprocessing [Электронный ресурс] / Luciano Ramalho. – Режим доступа: <https://www.youtube.com/watch?v=PdMJlXgWoQ> (дата обращения: 12.05.2026).
5. Гражданский кодекс Российской Федерации (часть первая) [Текст] : федер. закон от 30.11.1994 № 51-ФЗ : (в ред. от 25.02.2022) : ст. 447–449 «Заключение договора на торгах» // Собрание законодательства РФ. – 1994. – № 32. – Ст. 3301.
6. О защите прав потребителей [Текст] : закон Российской Федерации от 07.02.1992 № 2300-1 : (в ред. от 11.06.2021) // Ведомости Съезда народных депутатов РФ и Верховного Совета РФ. – 1992. – № 15. – Ст. 766.
7. О персональных данных [Текст] : федер. закон от 27.07.2006 № 152-ФЗ : (в ред. от 14.07.2022) // Собрание законодательства РФ. – 2006. – № 31 (ч. I). – Ст. 3451.
8. FastAPI Documentation : Python Web Framework [Электронный ресурс] / Sebastián Ramírez. – Режим доступа: <https://fastapi.tiangolo.com> (дата обращения: 12.05.2026).
9. SQLAlchemy 2.0 Documentation [Электронный ресурс] / SQLAlchemy Team. – Режим доступа: <https://docs.sqlalchemy.org/en/20> (дата обращения: 12.05.2026).

10. asyncpg Documentation : A fast PostgreSQL Database Client Library for Python/asyncio [Электронный ресурс] / MagicStack Inc. – Режим доступа: <https://magicstack.github.io/asyncpg> (дата обращения: 12.05.2026).

11. PostgreSQL 14 Documentation [Электронный ресурс] / PostgreSQL Global Development Group. – Режим доступа: <https://www.postgresql.org/docs/14> (дата обращения: 12.05.2026).

12. Фаулер, М. Шаблоны корпоративных приложений [Текст] = Patterns of Enterprise Application Architecture / Мартин Фаулер ; пер. с англ. – Москва : Вильямс, 2017. – 544 с.

13. Ное, М. Asynchronous Tasks with FastAPI and Celery vs BackgroundTasks vs Async [Электронный ресурс] / TestDriven.io. – Режим доступа: <https://testdriven.io/blog/fastapi-and-celery> (дата обращения: 12.05.2026).

14. Дейт, К. Дж. Введение в системы баз данных [Текст] = An Introduction to Database Systems / К. Дж. Дейт ; пер. с англ. К. А. Птицына. – 8-е изд. – Москва : Вильямс, 2015. – 1328 с.

15. Клеппман, М. Высоконагруженные приложения. Программирование, масштабирование, поддержка [Текст] = Designing Data-Intensive Applications / Мартин Клеппман ; пер. с англ. – Санкт-Петербург : Питер, 2018. – 640 с.

16. Fielding, R. HTTP Semantics : RFC 9110 [Электронный ресурс] / R. Fielding, M. Nottingham, J. Reschke ; IETF. – 2022. – 226 p. – Режим доступа: <https://datatracker.ietf.org/doc/html/rfc9110> (дата обращения: 12.05.2026).

17. Jones, M. JSON Web Token (JWT) : RFC 7519 [Электронный ресурс] / M. Jones, J. Bradley, N. Sakimura ; IETF. – 2015. – 30 p. – Режим доступа: <https://datatracker.ietf.org/doc/html/rfc7519> (дата обращения: 12.05.2026).

18. Pydantic Documentation : Data validation using Python type hints [Электронный ресурс] / Samuel Colvin et al. – Режим доступа: <https://docs.pydantic.dev> (дата обращения: 12.05.2026).

19. Fette, I. The WebSocket Protocol : RFC 6455 [Электронный ресурс] / I. Fette, A. Melnikov ; Internet Engineering Task Force (IETF). – 2011. – 71 p. – Режим доступа: <https://datatracker.ietf.org/doc/html/rfc6455> (дата обращения: 12.05.2026).

20. Python 3.12 documentation : asyncio – Asynchronous I/O [Электронный ресурс] / Python Software Foundation. – Режим доступа: <https://docs.python.org/3/library/asyncio.html> (дата обращения: 12.05.2026).

21. Advisory Locks in PostgreSQL [Электронный ресурс] / PostgreSQL Wiki. – Режим доступа: https://wiki.postgresql.org/wiki/Advisory_Locks (дата обращения: 12.05.2026).

22. Richardson, C. Pattern: Transactional outbox [Электронный ресурс] / Chris Richardson // Microservices.io. – Режим доступа: <https://microservices.io/patterns/data/transactional-outbox.html> (дата обращения: 12.05.2026).

23. Explicit Locking : Section 13.3 of PostgreSQL 14 Documentation [Электронный ресурс] / PostgreSQL Global Development Group. – Режим доступа: <https://www.postgresql.org/docs/14/explicit-locking.html> (дата обращения: 12.05.2026).

24. Alembic Documentation : Database Migrations Tool for SQLAlchemy [Электронный ресурс] / Mike Bayer. – Режим доступа: <https://alembic.sqlalchemy.org> (дата обращения: 12.05.2026).

25. Молинаро, Э. SQL. Сборник рецептов [Текст] = SQL Cookbook / Энтони Молинаро ; пер. с англ. – 2-е изд. – Санкт-Петербург : Питер, 2021. – 624 с.

26. Грабер, М. SQL : практические приёмы [Текст] / Мартин Грабер ; пер. с англ. В. С. Гусарова. – Москва : Лори, 2014. – 354 с.

27. Лутц, М. Изучаем Python [Текст] : в 2 т. / Марк Лутц ; пер. с англ. – 5-е изд. – Москва : ДМК Пресс, 2020. – Т. 1: 832 с. ; Т. 2: 720 с.

28. Docker Engine Reference Documentation [Электронный ресурс] / Docker Inc. – Режим доступа: <https://docs.docker.com/engine> (дата обращения: 12.05.2026).

29. OWASP Application Security Verification Standard : Version 4.0.3 [Электронный ресурс] / OWASP Foundation. – 2022. – Режим доступа: <https://owasp.org/www-project-application-security-verification-standard> (дата обращения: 12.05.2026).

30. Argon2 : Memory-Hard Function for Password Hashing and Other Applications. RFC 9106 [Электронный ресурс] / A. Biryukov, D. Dinu, D. Khovratovich, S. Josefsson ; IETF. – 2021. – Режим доступа: <https://datatracker.ietf.org/doc/html/rfc9106> (дата обращения: 12.05.2026).

31. О мерах поддержки IT-отрасли [Электронный ресурс] : постановление Правительства РФ от 30.09.2022 № 1729 // Официальный интернет-портал правовой информации. – Электрон. дан. – Режим доступа: <http://publication.pravo.gov.ru> (дата обращения: 12.05.2026).

32. Налоговый кодекс Российской Федерации (часть вторая) [Текст] : федер. закон от 05.08.2000 № 117-ФЗ : подп. 26 п. 2 ст. 149 // Собрание законодательства РФ. – 2000. – № 32. – Ст. 3340.

33. ГОСТ 12.0.003-2015. Система стандартов безопасности труда. Опасные и вредные производственные факторы. Классификация [Текст]. – Введ. 2017-03-01. – Москва : Стандартинформ, 2016. – 16 с.

34. ГОСТ 12.2.032-78. Система стандартов безопасности труда. Рабочее место при выполнении работ сидя. Общие эргономические требования [Текст]. – Введ. 1979-01-01. – Москва : Изд-во стандартов, 1978. – 9 с.

35. ГОСТ Р 50923-96. Дисплеи. Рабочее место оператора. Общие эргономические требования и требования к производственной среде [Текст]. – Введ. 1997-07-01. – Москва : Изд-во стандартов, 1996. – 22 с.

36. СП 52.13330.2016. Свод правил. Естественное и искусственное освещение. Актуализированная редакция СНиП 23-05-95* [Текст]. – Введ. 2017-05-08. – Москва : Минстрой России, 2016. – 124 с.

37. СанПиН 1.2.3685-21. Гигиенические нормативы и требования к обеспечению безопасности и (или) безвредности для человека факторов среды обитания [Текст] : утв. постановлением Главного государственного санитарного врача РФ от 28.01.2021 № 2. – Москва, 2021. – 469 с.

38. СП 12.13130.2009. Определение категорий помещений, зданий и наружных установок по взрывопожарной и пожарной опасности [Текст]. – Введ. 2009-05-01. – Москва : ВНИИПО МЧС России, 2009. – 28 с.

39. ГОСТ 12.1.038-82. Система стандартов безопасности труда. Электробезопасность. Предельно допустимые значения напряжений прикосновения и токов [Текст]. – Введ. 1983-07-01. – Москва : Изд-во стандартов, 1982. – 7 с.

40. Об отходах производства и потребления [Текст] : федер. закон от 24.06.1998 № 89-ФЗ : (в ред. от 14.07.2022) // Собрание законодательства РФ. – 1998. – № 26. – Ст. 3009.

41. Гамма, Э. Приёмы объектно-ориентированного проектирования. Паттерны проектирования [Текст] = Design Patterns: Elements of Reusable Object-Oriented Software / Эрих Гамма, Ричард Хелм, Ральф Джонсон, Джон Влиссидес ; пер. с англ. – Санкт-Петербург : Питер, 2020. – 368 с.

ПРИЛОЖЕНИЕ Г

Г.1. `app/services/scheduler_election.py` – выбор ведущего узла-планировщика

Листинг Г.1 – Получение рекомендательной блокировки на стороне PostgreSQL

```
async def try_become_scheduler_leader() -> bool:
    """Try to claim the scheduler leader lock via pg_try_advisory_lock."""
    global _leader_connection
    if not _election_enabled():
        return True
    if _leader_connection is not None:
        return True

    conn = await _db_module.engine.connect()
    try:
        result = await conn.execute(
            text("SELECT pg_try_advisory_lock(:key)",
                {"key": SCHEDULER_LOCK_KEY},
            )
        )
        got = bool(result.scalar_one())
    except Exception:
        logger.exception("scheduler leader election: query failed")
        await conn.close()
        return False

    if not got:
        await conn.close()
        return False

    _leader_connection = conn
    return True
```

Г.2. `app/services/email_outbox.py` – воркер очереди писем

Листинг Г.2 – Выборка партии писем: чтение id и построчный захват FOR UPDATE SKIP LOCKED

```
async def _run_one_tick() -> int:
    """Обработать одну партию писем; вернуть число обработанных строк."""
    # Лёгкое чтение id-кандидатов без блокировки строк. Порядок –
    # FIFO по created_at, чтобы письмо-подтверждение ушло раньше
    # уведомления, если оба попали в одну секунду.
    async with SessionLocal() as db:
        candidate_ids = (await db.execute(
            select(EmailOutbox.id)
            .where(EmailOutbox.status == "pending")
            .where(EmailOutbox.next_attempt_at <= utcnow())
            .order_by(EmailOutbox.created_at)
            .limit(WORKER_BATCH_SIZE)
        )).scalars().all()

    # Каждая строка – в своей сессии: блокировка FOR UPDATE живёт
    # ровно до её commit. Построчный захват через SKIP LOCKED даёт
    # горизонтальное масштабирование – воркеры не блокируют друг
```

```

# друга, проигравший просто пропускает строку. Прежняя схема с
# одним FOR UPDATE на всю партию при первом commit отпускала
# все блокировки разом и допускала двойную отправку письма.
for outbox_id in candidate_ids:
    async with SessionLocal() as db:
        row = (await db.execute(
            select(EmailOutbox)
            .where(EmailOutbox.id == outbox_id)
            .where(EmailOutbox.status == "pending")
            .with_for_update(skip_locked=True)
        )).scalar_one_or_none()
        if row is None:
            continue # терминальный статус или строку держит сосед
        await _process_one(row, db)
        await db.commit()

```

Г.3. app/routers/bids.py – атомарная обработка ставки

Листинг Г.3 – Размещение ставки под блокировкой строки SELECT FOR UPDATE

```

@router.post("/bids")
@limiter.limit("60/minute")
async def place_bid(
    request: Request,
    bid: BidCreate,
    current_user: User = Depends(require_verified_user),
    db: AsyncSession = Depends(get_db),
):
    bid_amount = to_decimal(bid.amount)

    # FOR UPDATE: конкурентные ставки на один лот выстраиваются
    # в очередь на стороне PostgreSQL - проверка и запись атомарны.
    auction = (
        await db.execute(
            select(Auction).where(Auction.id ==
bid.auction_id).with_for_update()
        )
    ).scalar_one_or_none()
    if not auction or not auction.is_active:
        raise HTTPException(status_code=400, detail="Аукцион недоступен")

    # Блокировка пользователя по id защищает от over-commit:
    # две параллельные ставки одного юзера на разные лоты не пройдут
    # проверку баланса независимо друг от друга.
    await lock_users_by_id(db, current_user.id)
    committed = await effective_committed_balance(
        db, current_user.id, bid.auction_id, auction.current_price
    )
    if current_user.balance - committed < bid_amount:
        raise HTTPException(status_code=400, detail="Недостаточно средств")

    db_bid = Bid(amount=bid_amount, user_id=current_user.id,
        auction_id=bid.auction_id)
    auction.current_price = bid_amount
    db.add(db_bid)
    await db.commit()

```

Г.4. app/services/auction_scheduler.py – защита от снайперских ставок

Листинг Г.4 – Автоматическое продление лота при поздней ставке

```
# В app/services/auction_scheduler.py:
ANTISNIPING_WINDOW = timedelta(seconds=120)
ANTISNIPING_EXTEND = timedelta(seconds=120)
MAX_ANTISNIPING_EXTENSIONS = 30 # потолок цепочки ~ 1 час

# В обработчике POST /bids после успешной валидации:
extended = False
now = utcnow()
# Нижняя граница delta > 0: если ставка проскочила проверку
# истёкшего дедлайна на микросекунды (гонка с тиком планировщика),
# end_time - now отрицательна, и продлевать завершившийся лот нельзя.
delta = auction.end_time - now
if (timedelta(0) < delta < auction_scheduler.ANTISNIPING_WINDOW
    and auction.extensions_count
    < auction_scheduler.MAX_ANTISNIPING_EXTENSIONS):
    # Ставка в окне закрытия и кэп не исчерпан - продлеваем end_time
    # на полное окно от текущего момента (не от старого end_time).
    auction.end_time = now + auction_scheduler.ANTISNIPING_EXTEND
    auction.ending_soon_notified = False
    auction.extensions_count += 1
    extended = True

db.add(db_bid)
await db.commit()

if extended:
    # Отменяем старую сопрограмму asyncio.Task, которая всё ещё
    # спит до прежнего end_time, и заводим новую с обновлённым
    # дедлайном. Самовосстановление таймера дало бы такой же
    # эффект, но с лишним пробуждением.
    auction_scheduler.schedule_auction(auction)
```

Г.5. app/services/websocket_manager.py – широковещение подписчикам

Листинг Г.5 – Рассылка сообщения подписчикам с очисткой мёртвых соединений

```
async def _fan_out(self, registry: dict[int, list[WebSocket]],
                    key: int, message: dict):
    """Отправить message каждому сокету в registry[key].
    Соединения, на которых send_json бросил исключение,
    удаляются - без очистки реестр рос бы неограниченно."""
    bucket = registry.get(key)
    if not bucket:
        return
    # Snapshot bucket: каждое await send_json уступает event loop;
    # параллельный disconnect, вызывающий bucket.remove, сдвинул бы
    # индексы под живым итератором и пропустил бы рабочий сокет.
    snapshot = list(bucket)
    dead: list[WebSocket] = []
    for connection in snapshot:
        try:
```

```

        await connection.send_json(message)
    except Exception as exc:
        logger.debug("Dropping dead websocket on %s: %s", key, exc)
        dead.append(connection)
    for connection in dead:
        if connection in bucket:
            bucket.remove(connection)
    if not bucket:
        registry.pop(key, None)

async def broadcast(self, message: dict, auction_id: int):
    # Широковещание подписчикам страницы лота: новая ставка,
    # обновлённое end_time при срабатывании anti-sniping и т.д.
    await self._fan_out(self.active_connections, auction_id, message)

async def send_notification(self, user_id: int, message: dict):
    # Канал уведомлений конкретного пользователя -
    # сообщение о перебитой ставке, выигрыше, продаже.
    await self._fan_out(self.user_connections, user_id, message)

```

ПРИЛОЖЕНИЕ Д

Д.1. Регистрация и вход в систему

На главной странице сервиса в правом верхнем углу нажмите кнопку «Войти». В открывшемся окне переключитесь на вкладку «Регистрация» (рисунок Е.2). Заполните три поля: имя пользователя (от трёх до тридцати двух символов; разрешены латиница, кириллица, цифры, подчёркивание и дефис), адрес электронной почты, пароль (от восьми символов). После нажатия кнопки «Зарегистрироваться» на указанный адрес уходит письмо со ссылкой подтверждения. Аккаунт считается активированным после перехода по ссылке. Без подтверждения адреса размещение лотов и ставок недоступно.

Для повторного входа используется вкладка «Вход» того же окна. При утере пароля воспользуйтесь ссылкой «Забыли пароль?» - на адрес уйдёт письмо с временной ссылкой восстановления. Защита от подбора пароля работает в две стадии: ограничение десять попыток в минуту с одного IP-адреса и блокировка конкретного аккаунта после пяти подряд неудачных входов на одну минуту с удвоением окна на каждой следующей серии (пять, десять, пятнадцать, двадцать неудач → одна минута, пять минут, пятнадцать минут, один час).

Д.2. Просмотр каталога лотов

Главная страница сервиса (рисунок Е.1) показывает плитку активных лотов с превью изображения, текущей ценой, временем до окончания торгов и именем продавца. В левой колонке доступны фильтры: статус (активные, завершённые, все), категория, тип аукциона (торги BID или покупка по фиксированной цене BIN), диапазон цены. Над плиткой расположена строка поиска по названию. По умолчанию лоты отсортированы по времени до окончания - первыми идут те, что скоро закроются.

Д.3. Размещение ставки

Клик по плитке лота открывает страницу торгов (рисунок Е.3). На странице показаны: фотографии лота, описание от продавца, текущая цена, история ставок, обратный отсчёт до закрытия. Справа расположен блок ставки: поле ввода суммы и кнопки быстрого приращения «+1», «+5», «+10», «+50», «+100». Минимально допустимая ставка - текущая цена плюс одна копейка. При нажатии «Поставить ставку» сумма резервируется на балансе, цена лота обновляется у всех пользователей, открывших страницу, в реальном времени по WebSocket.

Если ставка приходит в последние 120 секунд торгов, система автоматически продлевает их ещё на 120 секунд - всплывает уведомление «Лот продлён», таймер увеличивается. Каждая поздняя ставка снова продлевает торги, но не более тридцати раз (около часа): после этого ставки по-прежнему принимаются, а лот закрывается по расписанию.

Д.4. Создание собственного лота

Перед созданием лота требуется подтверждённый email и положительный баланс (пополнение - в личном кабинете). На главной странице нажмите «Создать лот» в верхней панели. В диалоге «Новый лот» (рисунок Е.5) заполните: название, описание, категорию, тип аукциона (торги BID или фиксированная цена BIN), стартовую цену, длительность в минутах. До пяти изображений добавляются перетаскиванием в левую часть формы; первое становится обложкой. После нажатия «Создать лот» торги начинаются немедленно и идут до истечения выбранной длительности.

Для собственных лотов на странице торгов вместо формы ставки отображается кнопка «Редактировать» (рисунок Е.4) - можно изменить описание и фотографии, пока на лот не сделана ни одна ставка. После первой ставки правки заблокированы.

Д.5. Управление аккаунтом

Личный кабинет (рисунок Е.6) открывается нажатием на имя пользователя в правом верхнем углу. Вкладка «Аккаунт» показывает имя пользователя, адрес электронной почты со статусом подтверждения, текущий баланс, дату регистрации; здесь же можно загрузить аватар. Загрузка изображений ограничена восемью мегабайтами на файл и пятьюдесятью мегабайтами в сумме за скользящие двадцать четыре часа на пользователя - при превышении квоты сервер возвращает ошибку с понятным текстом. Вкладка «Безопасность» содержит смену пароля и принудительный выход из всех сессий (механизм token_version - один UPDATE инвалидирует все выпущенные JWT). Вкладка «Уведомления» управляет рассылками по email; вкладка «Опасная зона» содержит удаление аккаунта.

В левой панели личного кабинета доступны: «Статистика» (сводка по выигранным/проданным лотам), «Баланс» (история пополнений и резервов), «Мои ставки», «Мои лоты», «Подписки», «Уведомления», «Значки» (достижения).

Д.6. Публичный профиль продавца

Имя продавца на странице лота - ссылка на его публичный профиль (рисунок Е.7). Публичный профиль показывает: аватар, имя пользователя, дату регистрации, число активных лотов и число выигранных торгов, плитку текущих активных лотов с возможностью быстрого перехода к каждому из них. Кнопка «Подписаться» включает рассылку уведомлений о новых лотах продавца на электронную почту. Под плиткой расположены отзывы покупателей по завершённым сделкам.

ПРИЛОЖЕНИЕ Е

В приложении приводятся снимки экранов работы веб-приложения «Лотус», иллюстрирующие основные пользовательские сценарии. Каждый снимок снабжён подписью с номером и описанием.

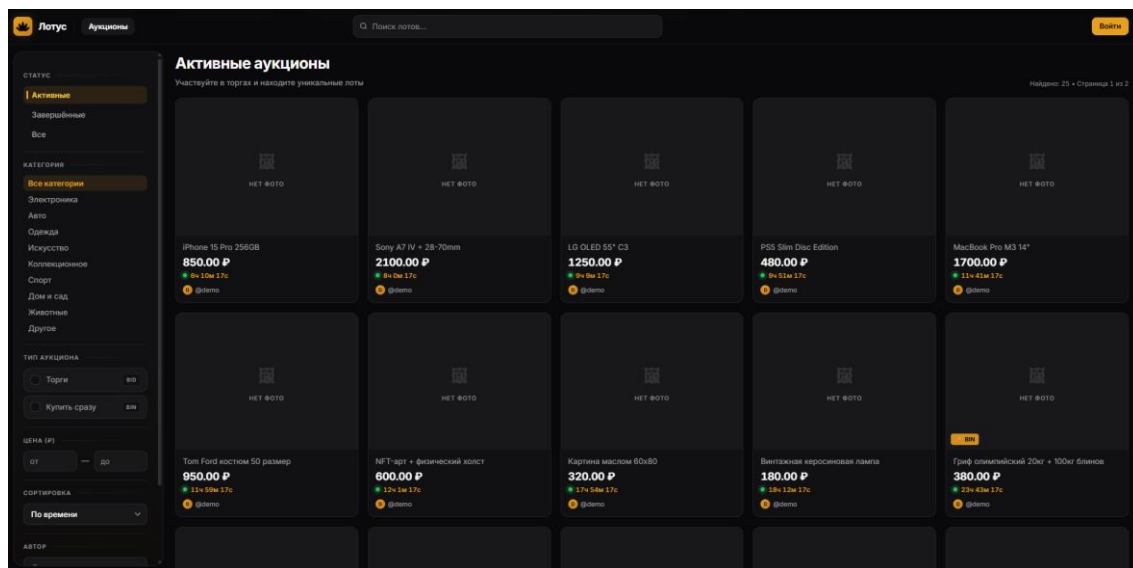


Рисунок Е.1 – Главная страница со списком активных лотов

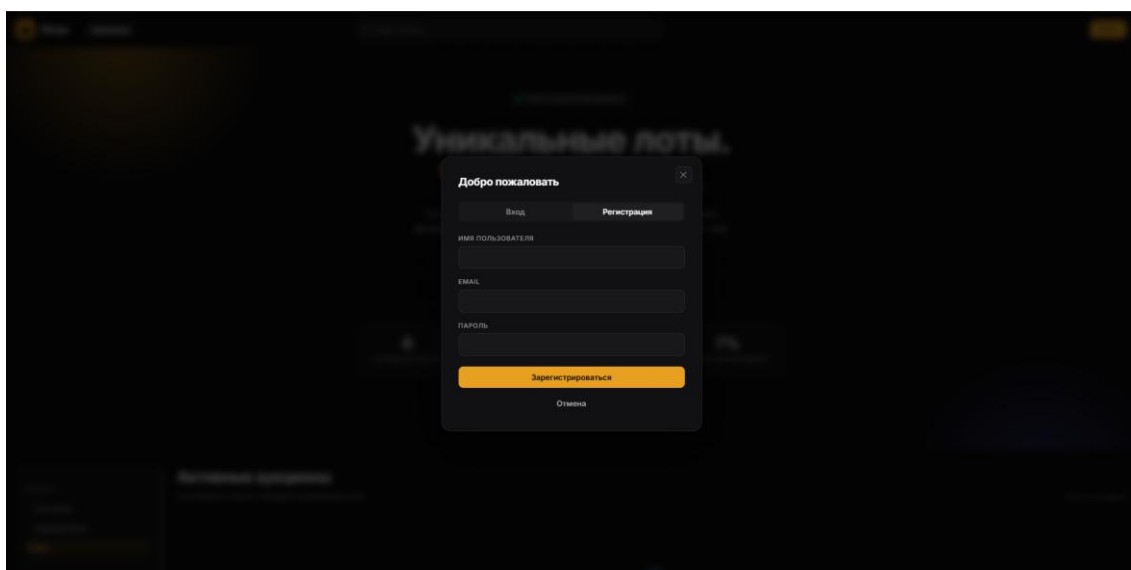


Рисунок Е.2 – Форма регистрации нового пользователя

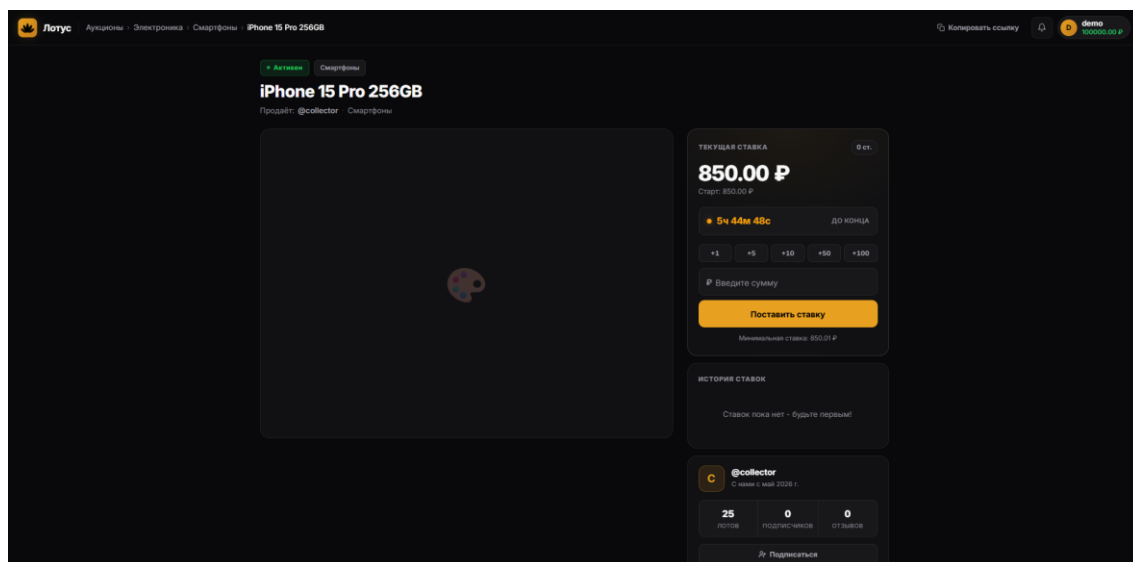


Рисунок Е.3 – Страница лота с точки зрения покупателя: ставки, таймер, история торгов

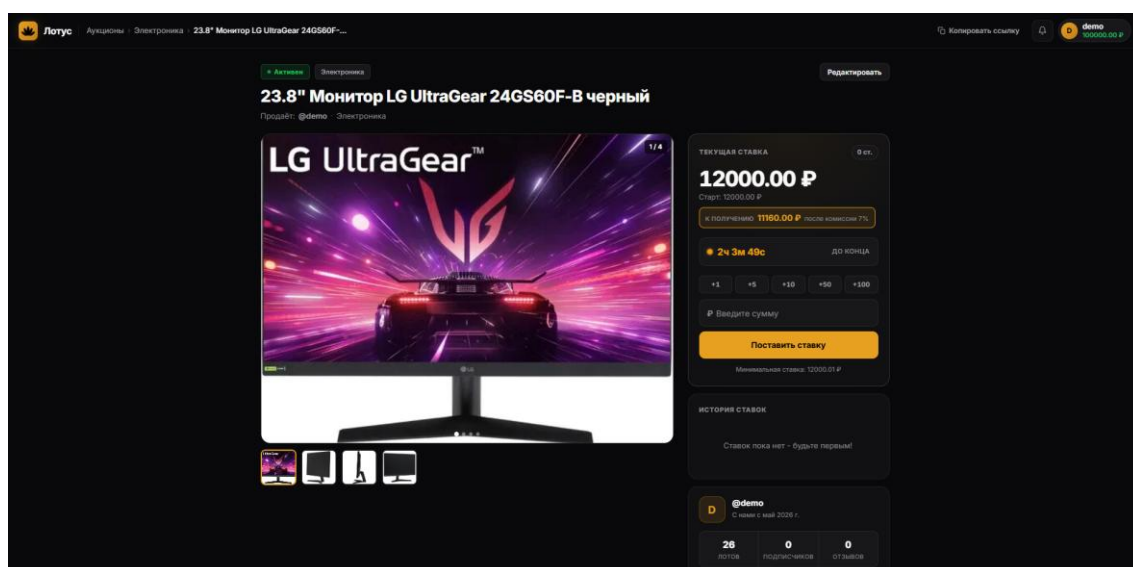


Рисунок Е.4 – Страница лота с точки зрения владельца (продавца): кнопка редактирования вместо ставки

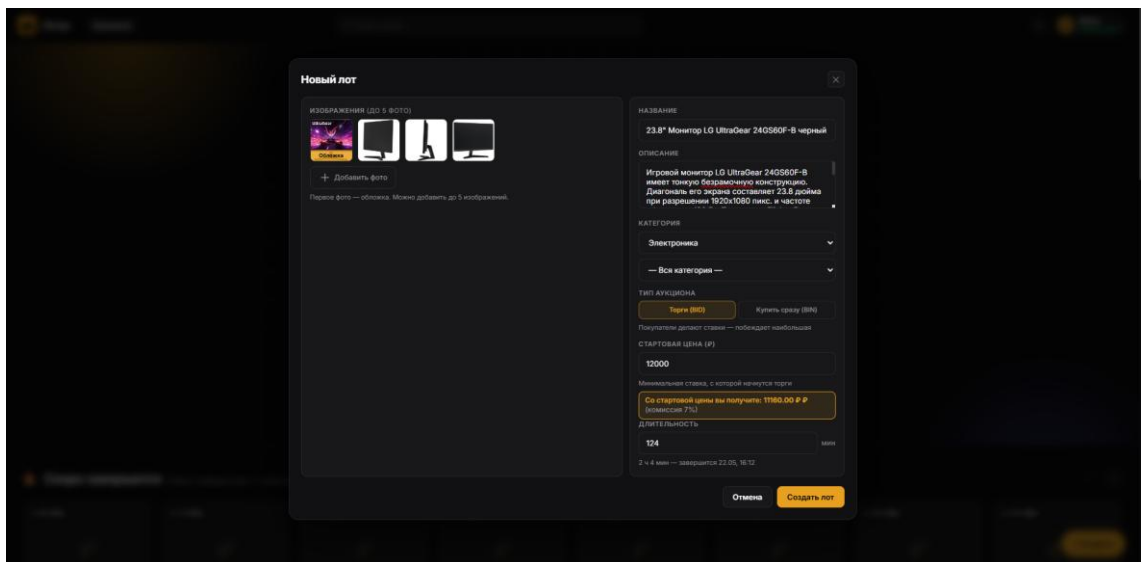


Рисунок Е.5 – Форма создания нового лота

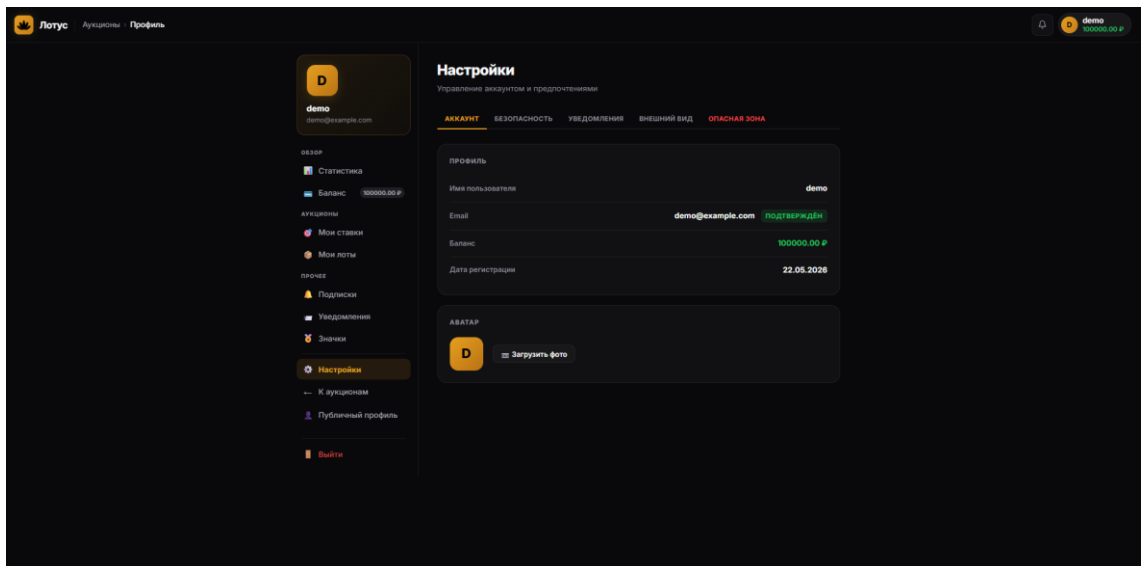


Рисунок Е.6 – Личный кабинет: настройки аккаунта, баланс, статус подтверждения адреса электронной почты

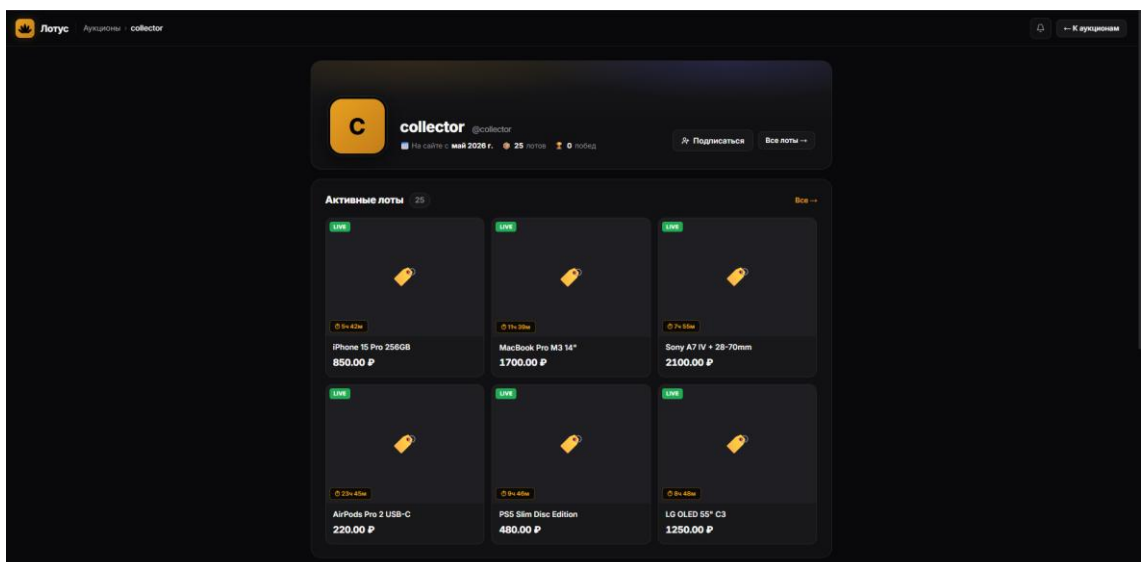


Рисунок Е.7 – Публичный профиль продавца со списком активных лотов